Autoencoders

Autoencoders blog post

AARHUS
UNIVERSITY

Dimensionality Reduction

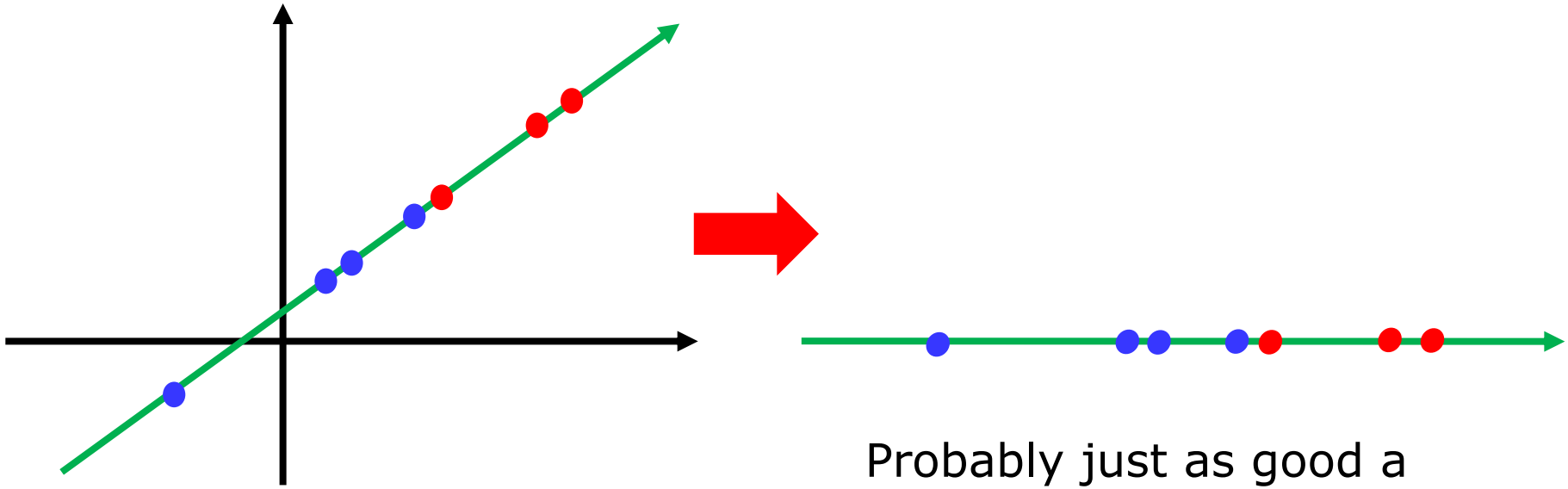
- Redundant Measurements:
  - Data may be redundant:
    - Car: Speed Mph, Fuel, Color, Seats, Speed km/h
  - Or highly correlated:
    - Student: Work Ethics, Grade



- Goal:
  - Remove redundancies.
  - Reduces number of features.
    - Speeds up/helps algorithms.
    - Visualization if very few features (e.g. 2 or 3).
    - Remove noise.

Redundancy

- 2 features but one-dimensional:



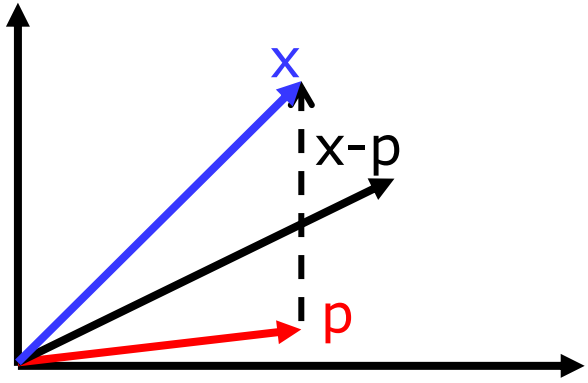
Probably just as good a representation for learning.

PCA

- Principal Component Analysis
  - Let $z_1,\ldots,z_k$ be k eigenvectors corresponding to largest eigenvalues of X^TX and project feature vector x to $p_x = \sum_{i=1}^{k}(x^T z_i)z_i$.
- Write x in the new basis:
  - $\hat{x} = \begin{pmatrix} x^T z_1 \\ \vdots \\ x^T z_k \end{pmatrix}$

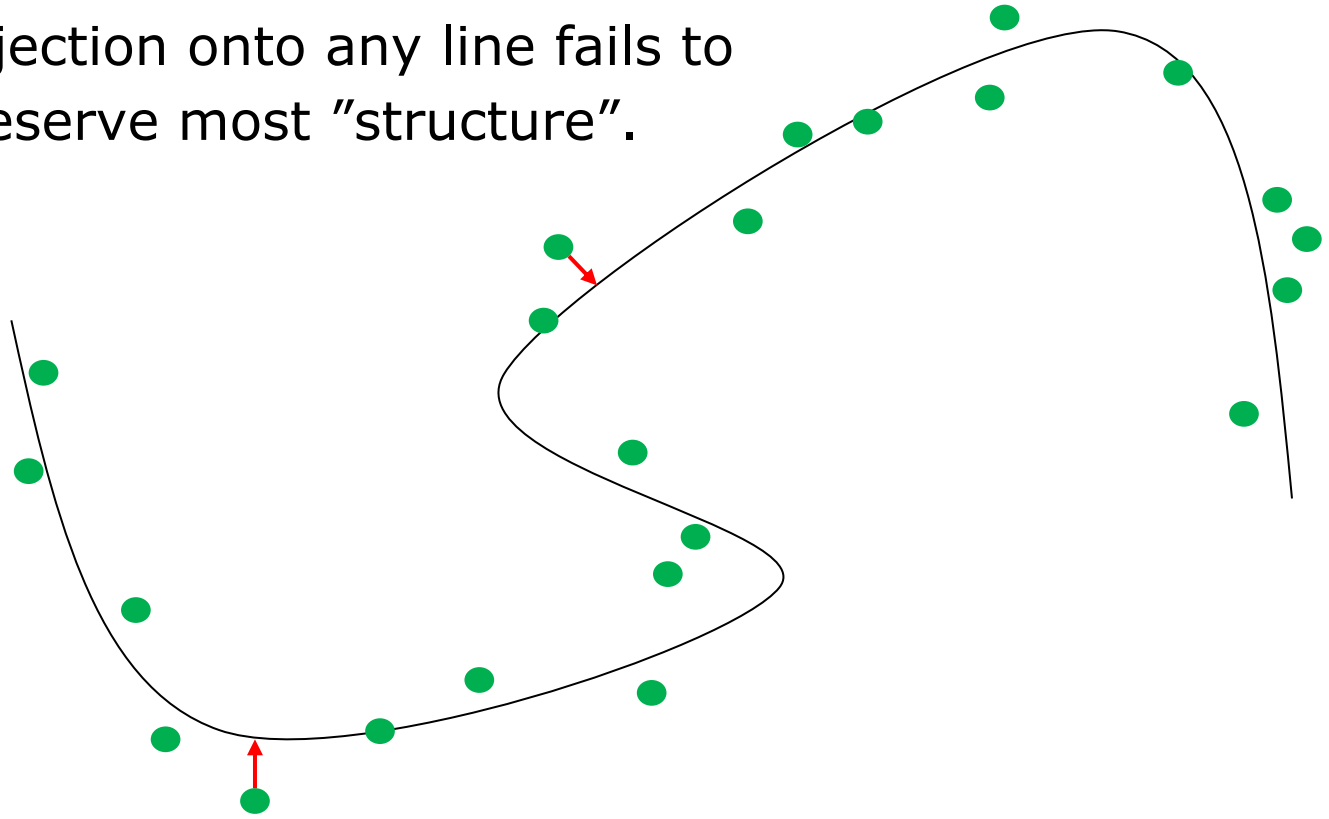


MNIST



PCA Limitations

- Embedding is linear
  - Projection onto any line fails to preserve most ”structure”.

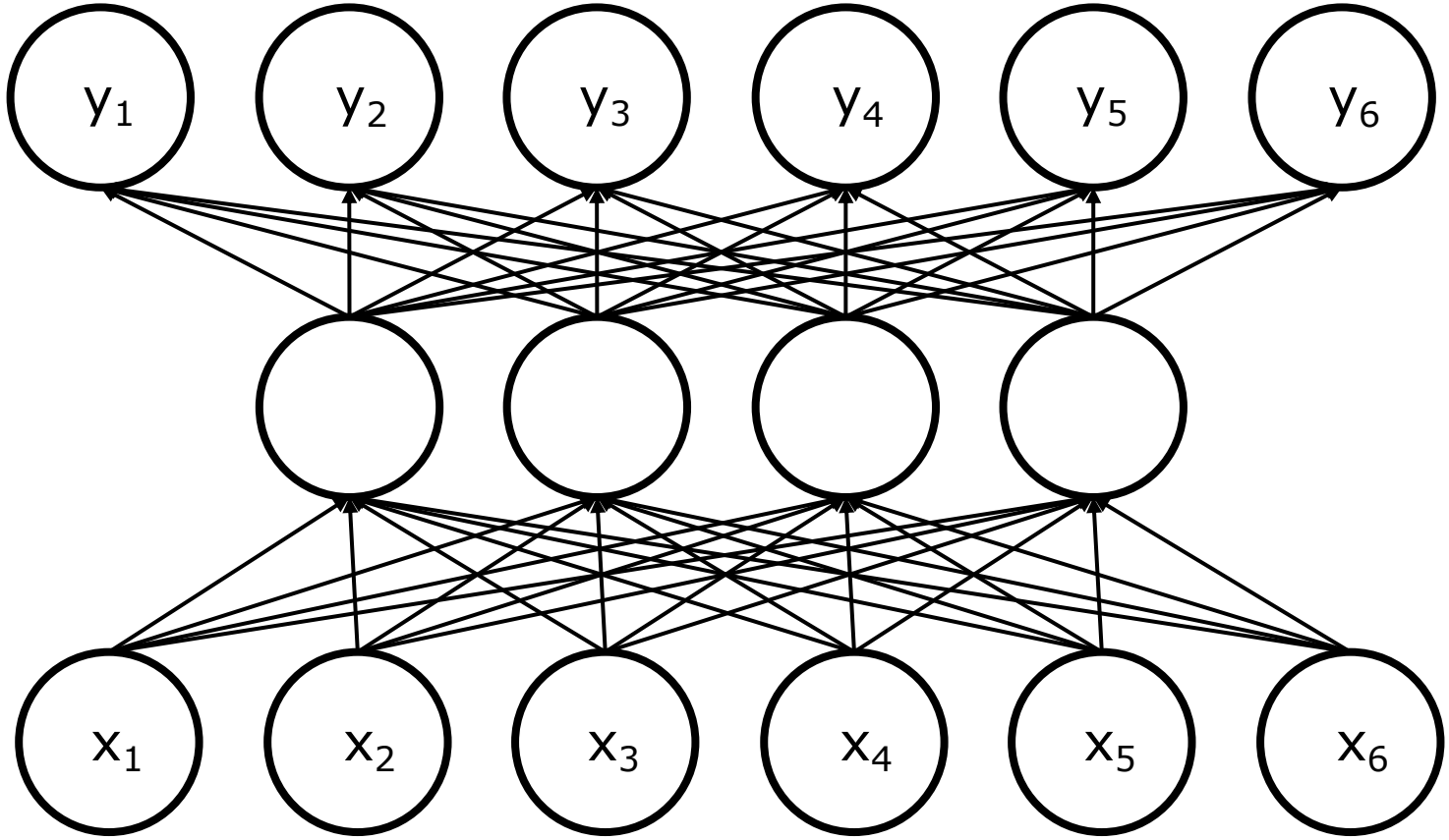


  - Non-linear embedding could be useful. Map to position along curve!

Autoencoders

- Neural Nets for Embeddings.

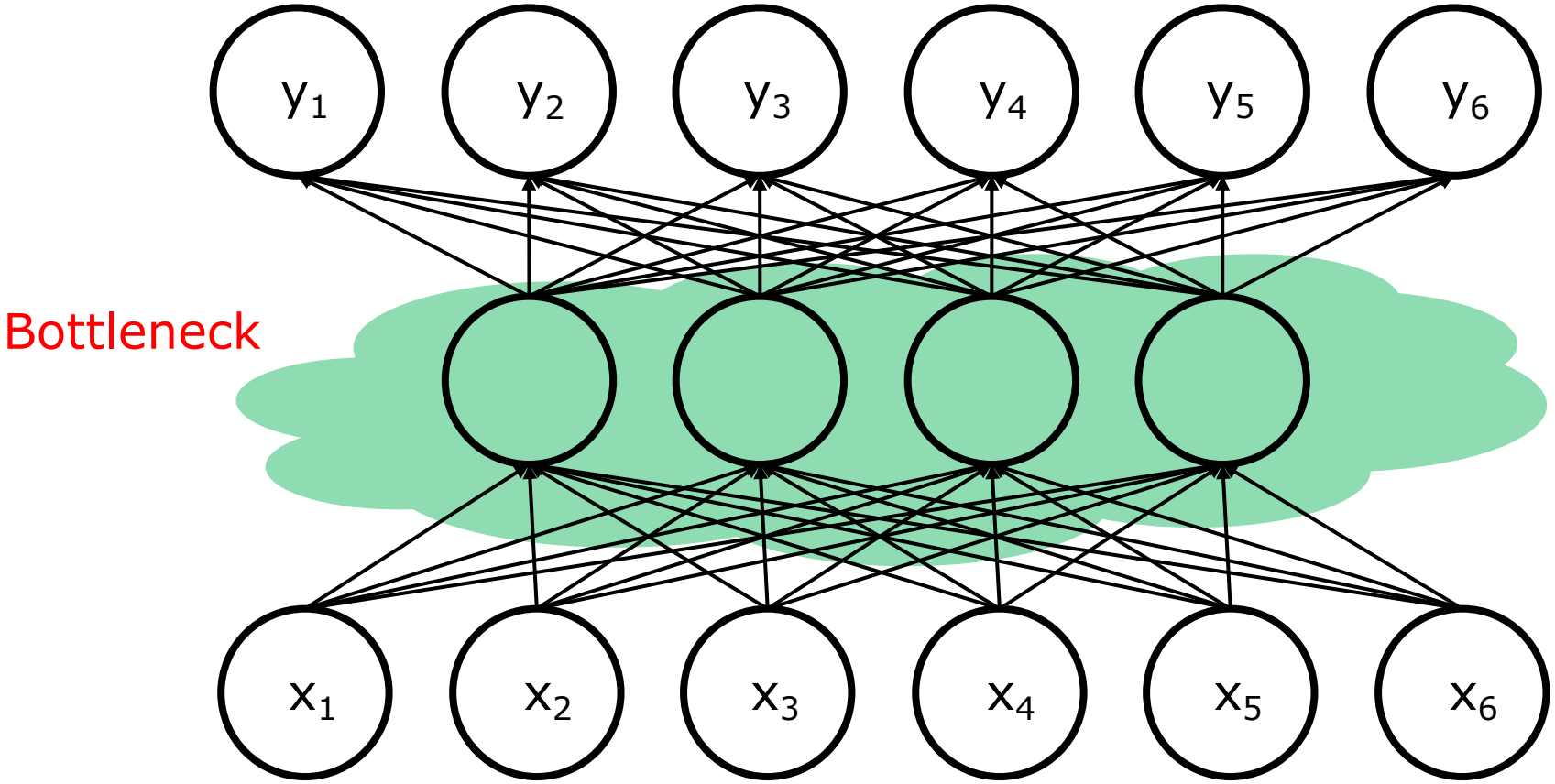
Idea: Want y=x



Autoencoders

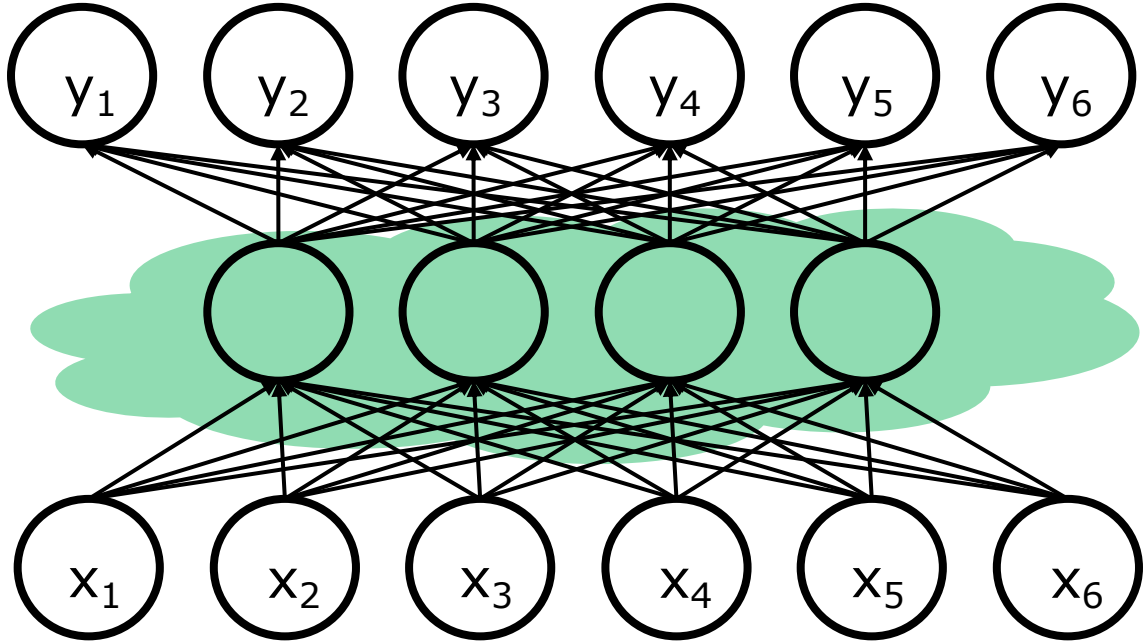
- Neural Nets for Embeddings.

Idea: Want y=x



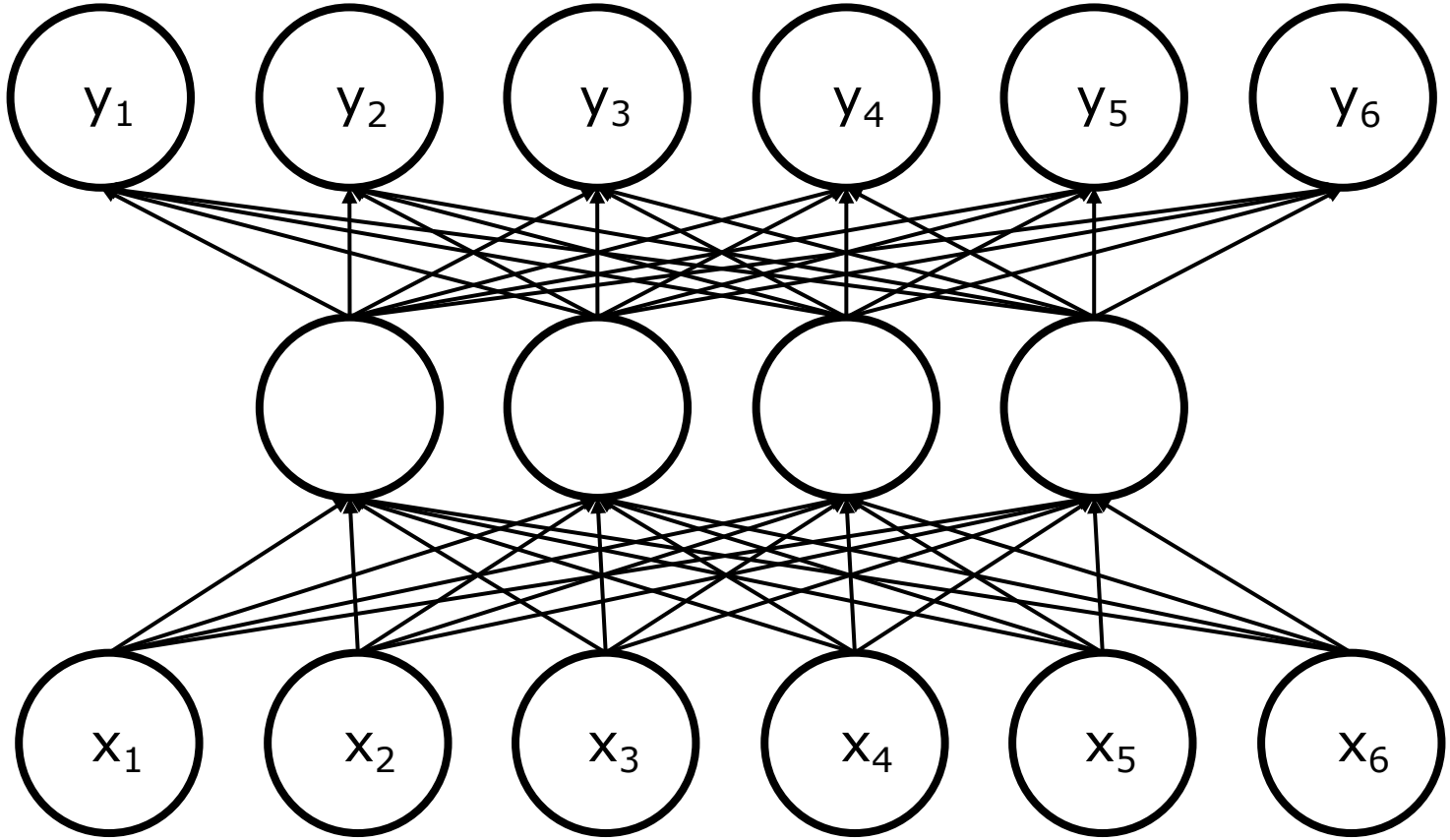
Autoencoders

- Neural Nets for Embeddings.
  - Intuition: If y=x then all information about x must be present in bottleneck layer.
  - Use output of bottleneck layer as features.



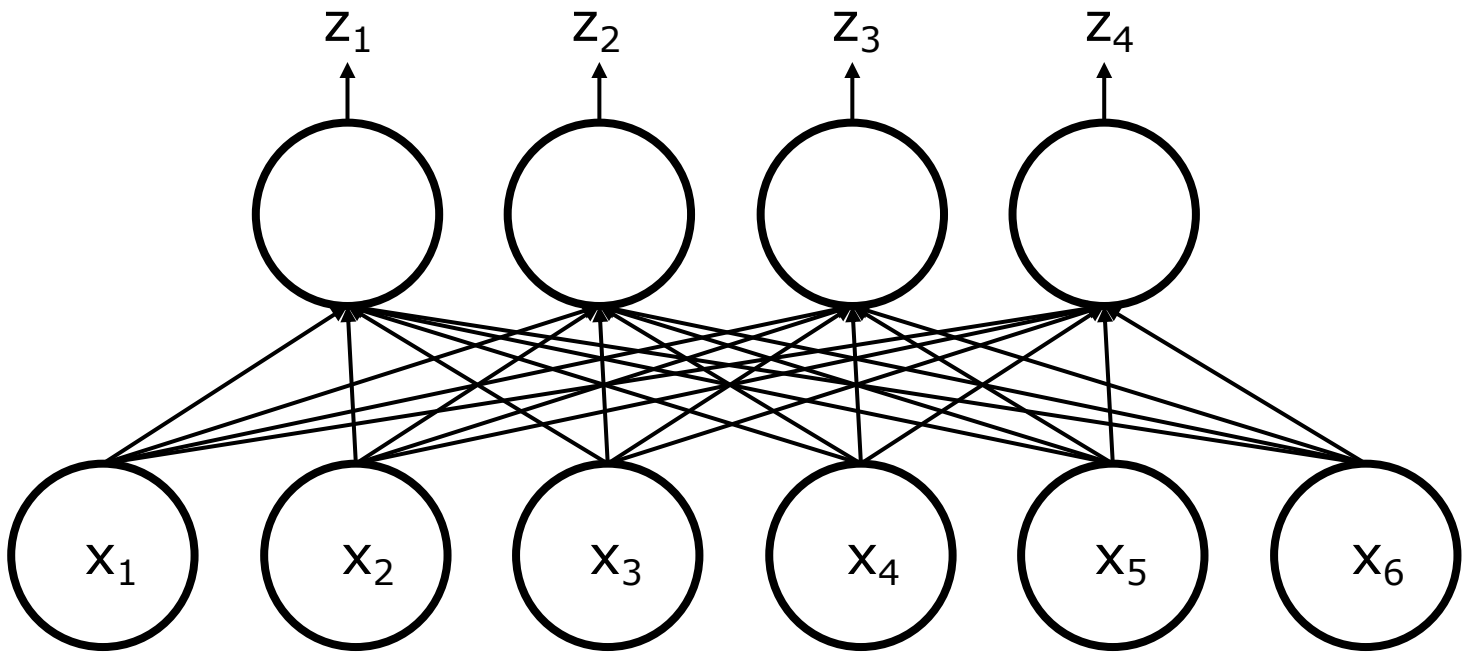
Autoencoders

- Embedding/Encoding x:



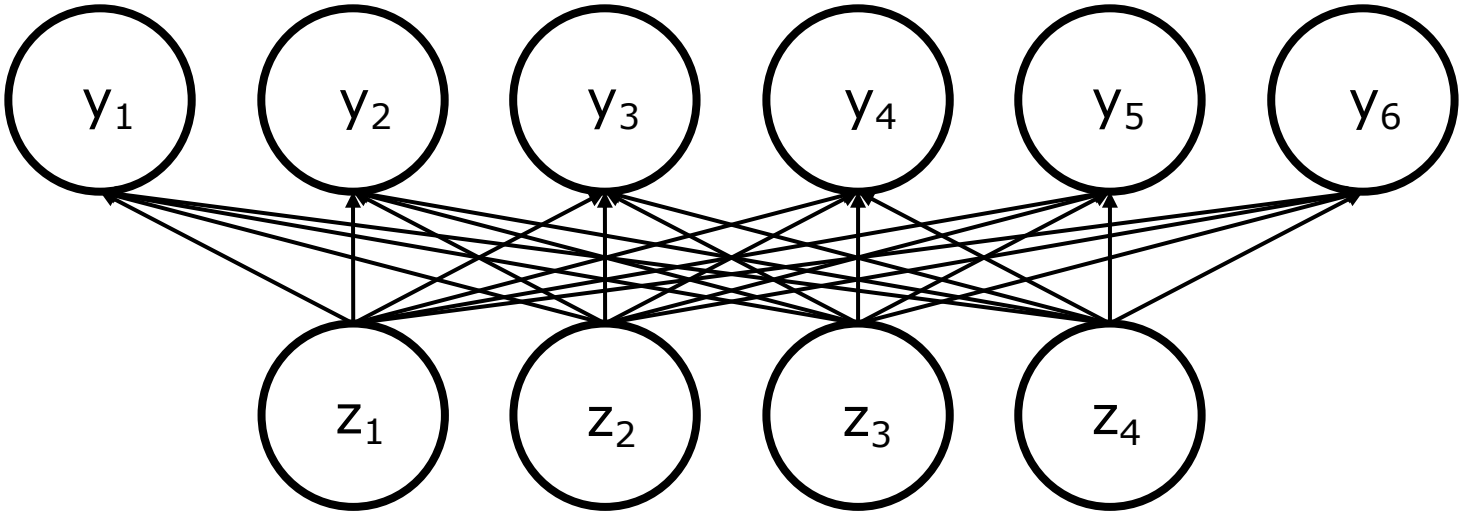
Autoencoders

- Embedding/Encoding x:
  - Just evaluate network until bottleneck layer.
    - Embed x to z.



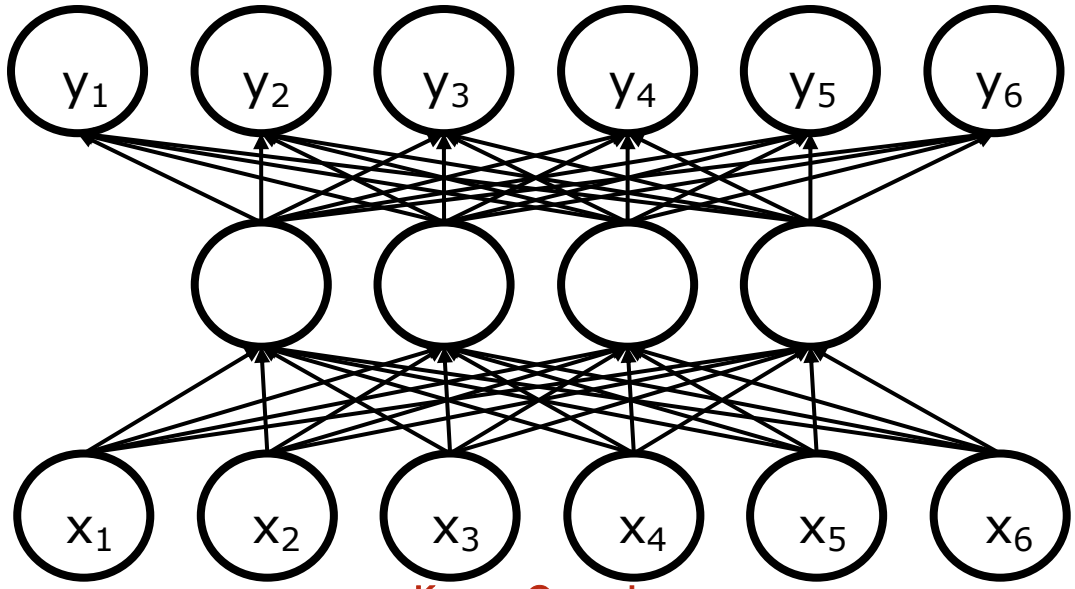
Autoencoders

- Recovering/Decoding x:
  - Given z, evaluate network after bottleneck layer to recover y=x.



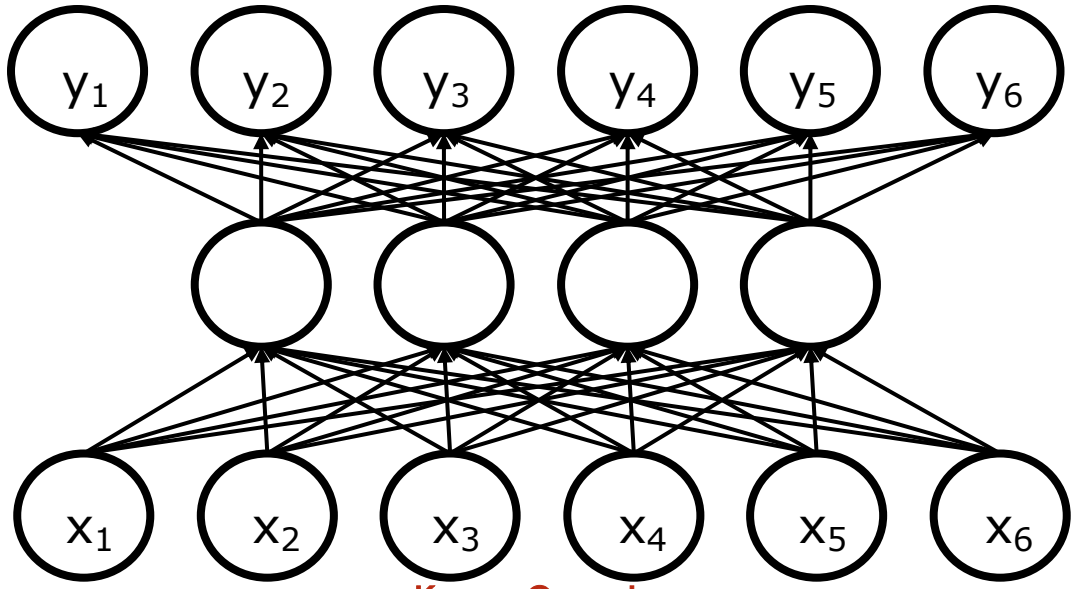
Autoencoders

- Training for y=x:
  - Like supervised learning, define pointwise loss
    - $L(x, y) = \sum_{i=1}^{d} (x_i - y_i)^2$
  - Minimize loss on training data: $\frac{1}{n} \sum_{j=1}^{n} L(x^j, ae(x^j))$
    - Here ae(x) is the output of the autoencoder.



Autoencoders

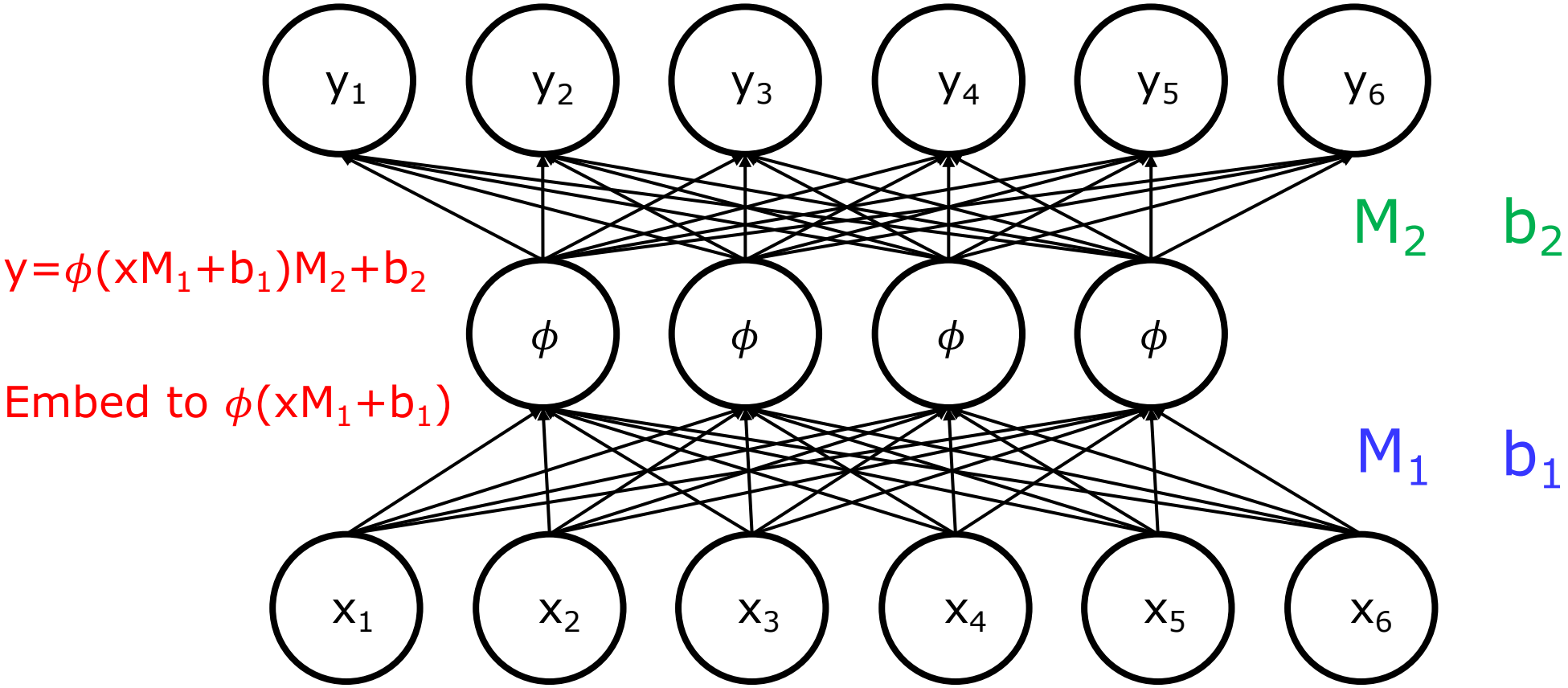
- Minimize
  - $\frac{1}{n}\sum_{j=1}^{n}\sum_{i=1}^{d}\left(x_i^j - ae(x^j)_i\right)^2$
- Simply use mini-batch stochastic gradient descent, with backpropagation to compute gradients.



Autoencoders

- Neural Nets for Embeddings.

Weight matrices, biases and activation functions.

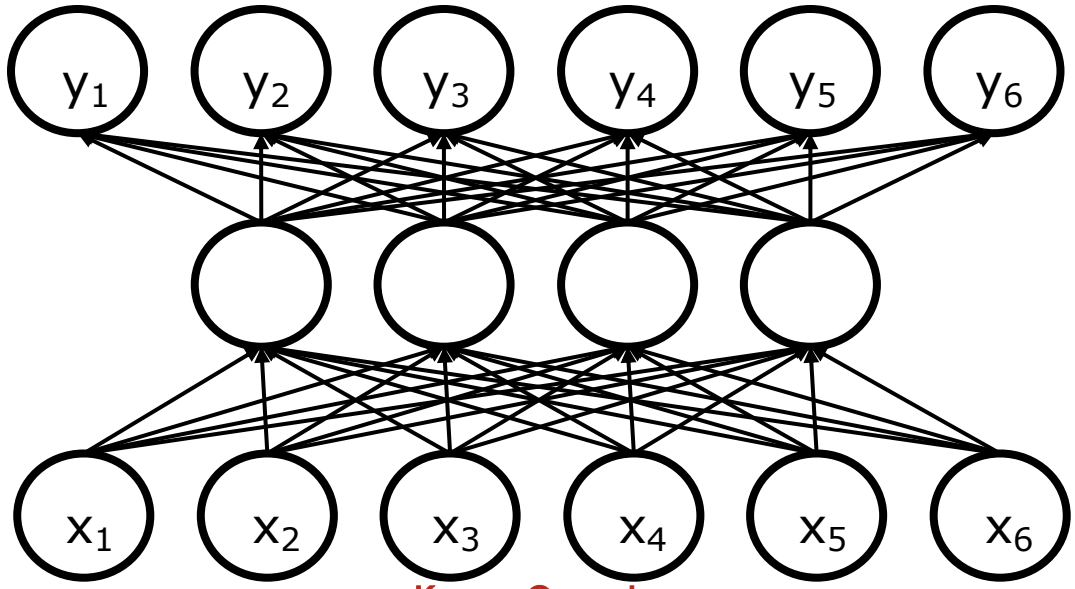


Autoencoders vs PCA

- PCA a special case:
  - Let Z be embedding matrix from PCA (top k eigenvectors of X^TX as columns).
  - Let $M_1=Z$, $M_2=Z^T$. No bias, Identity activation.

Same as PCA, we just treated vectors as column vectors in PCA.

x (as row vector) embedded to xZ



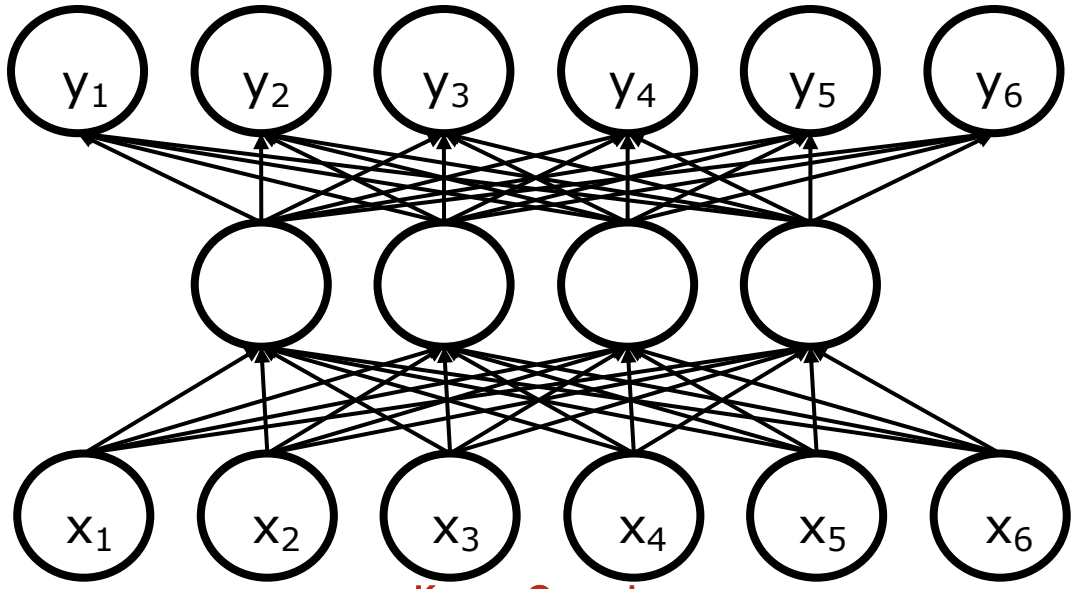
Gives projection of x written in basis Z.

Autoencoders vs PCA

- PCA a special case:
  - Let Z be embedding matrix from PCA (top k eigenvectors of X^TX as columns).
  - Let $M_1=Z$, $M_2=Z^T$. No bias, Identity activation.

Gives projection of x onto subspace spanned by Z

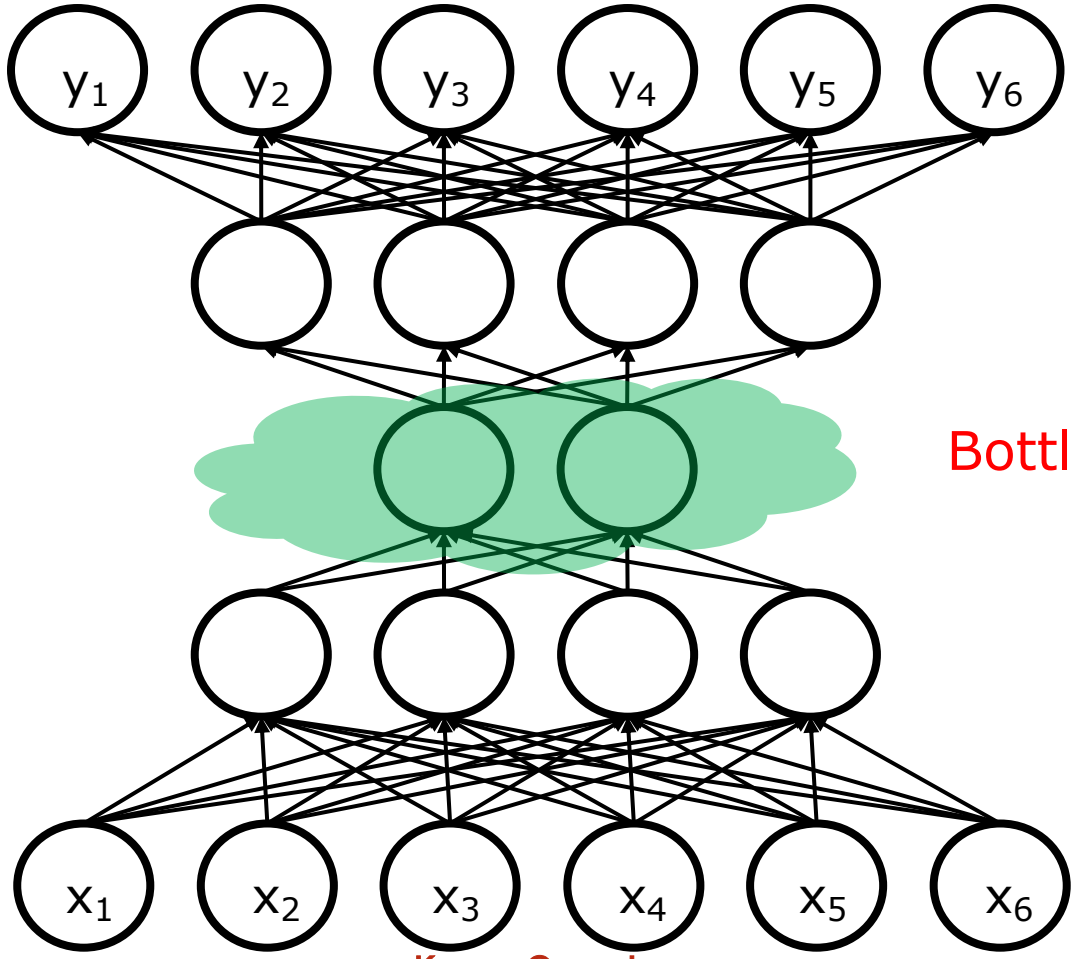
$y=xZZ^T$



PCA minimizes the same squared loss over all Z with unit length orthogonal columns.

Autoencoders

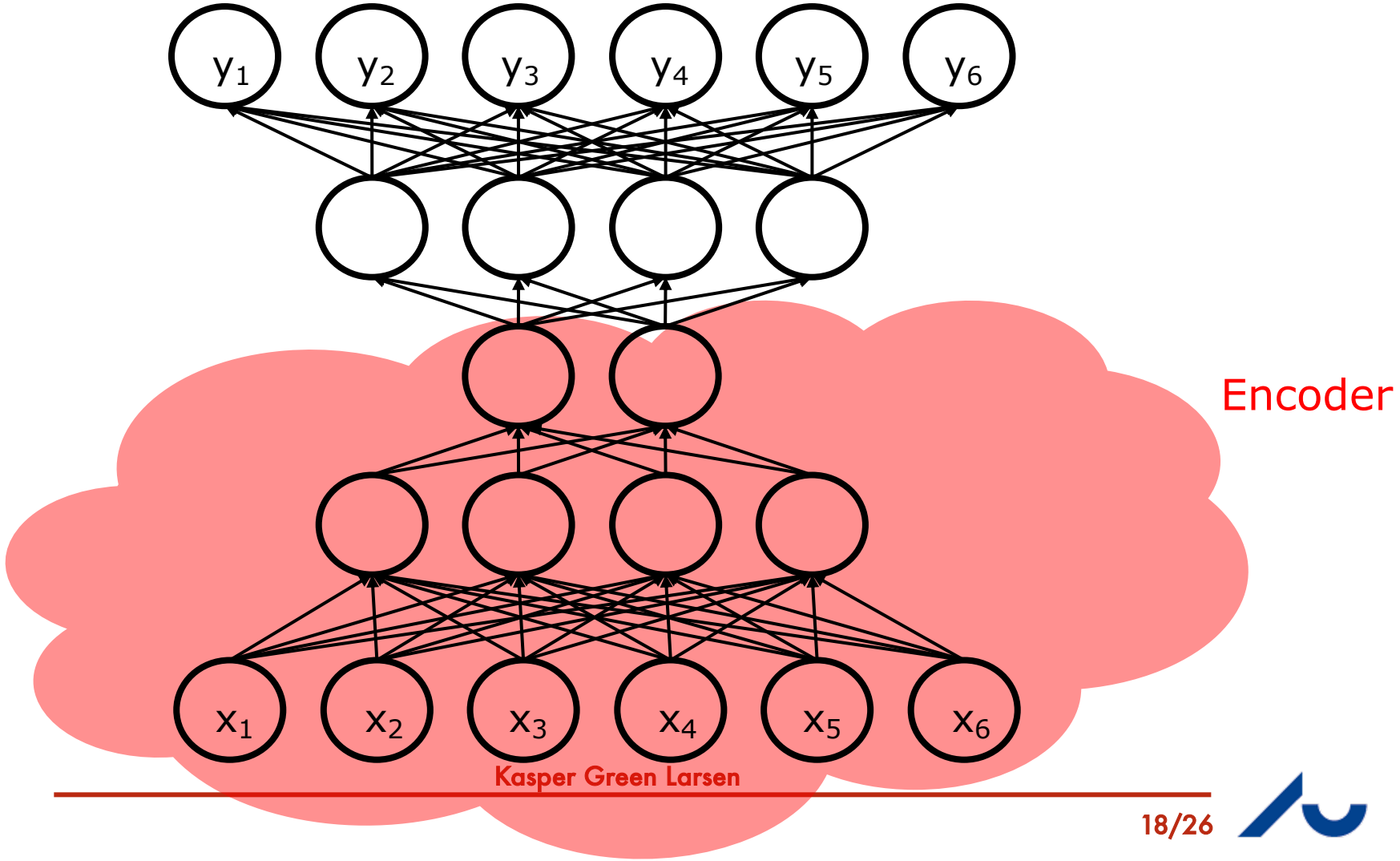
- Deeper Nets:



Bottleneck layer

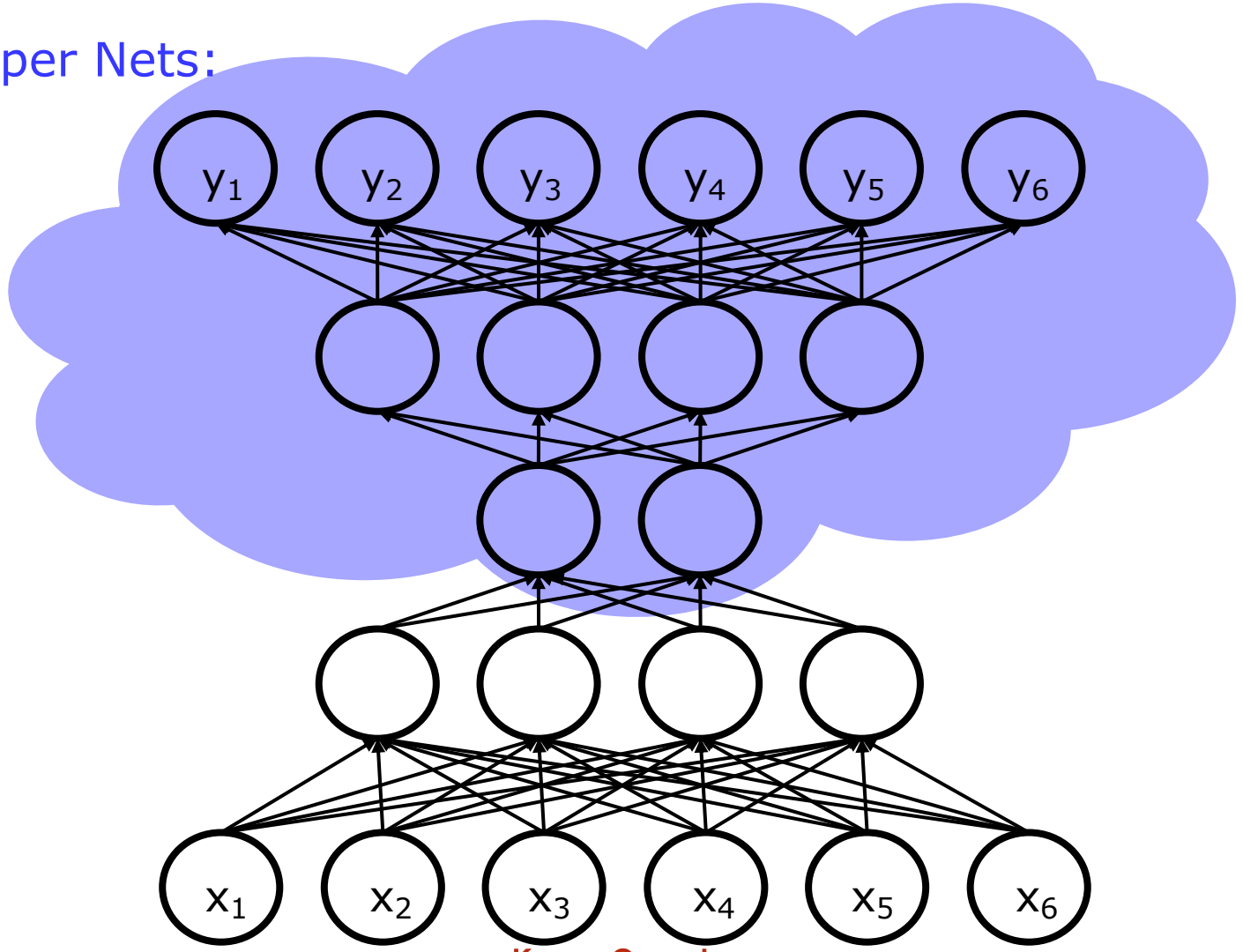
Autoencoders

- Deeper Nets:



Autoencoders

- Deeper Nets:



Decoder

Overfitting

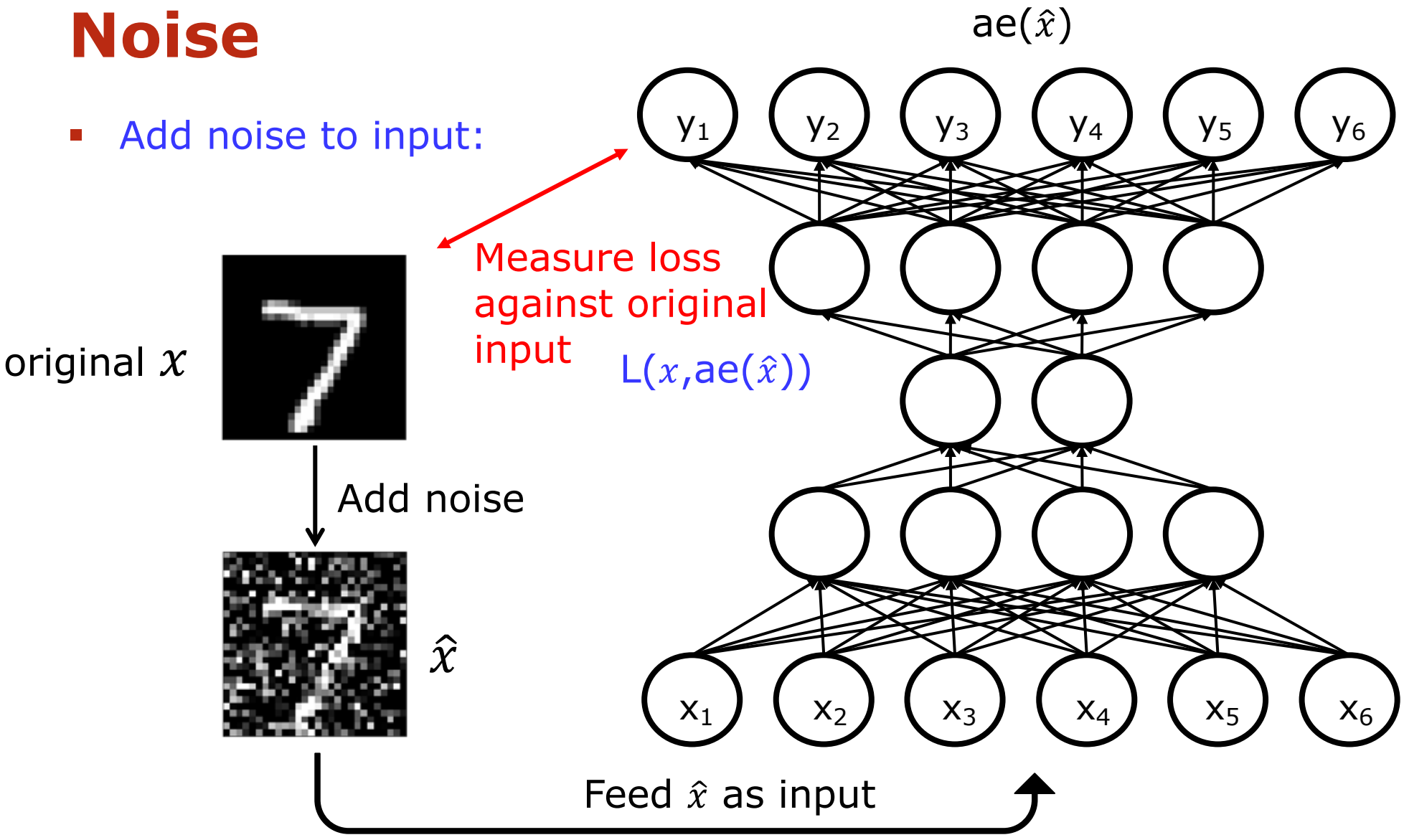
- Risk of overfitting data.
  - If little training data, then risk overfitting.
  - Works great on training data $x^1,\ldots,x^n$ but
    - On new item x, reconstruction very far from x.

- Approach:
  - Make insensitive to noise.



Noise

- Add noise to input:



Overfitting

- Risk of overfitting data.
  - If little training data, then risk overfitting.
  - Works great on training data $x^1,\ldots,x^n$ but
    - On new item x, reconstruction very far from x.

- Approach:
  - Make insensitive to noise.
    - Add noise to inputs.
    - Ask to recover original.
    - Intuition: Learns to extract the *actual* features of the data and ignore input noise.



Overfitting

- Risk of overfitting data.
  - If little training data, then risk overfitting.
  - Works great on training data $x^1,\ldots,x^n$ but
    - On new item x, reconstruction very far from x.

- Another approach:
  - Make *embedding* insensitive to small changes in input
    - In contrast to making *output* insensitive like we just did.
  - How can we formalize this?

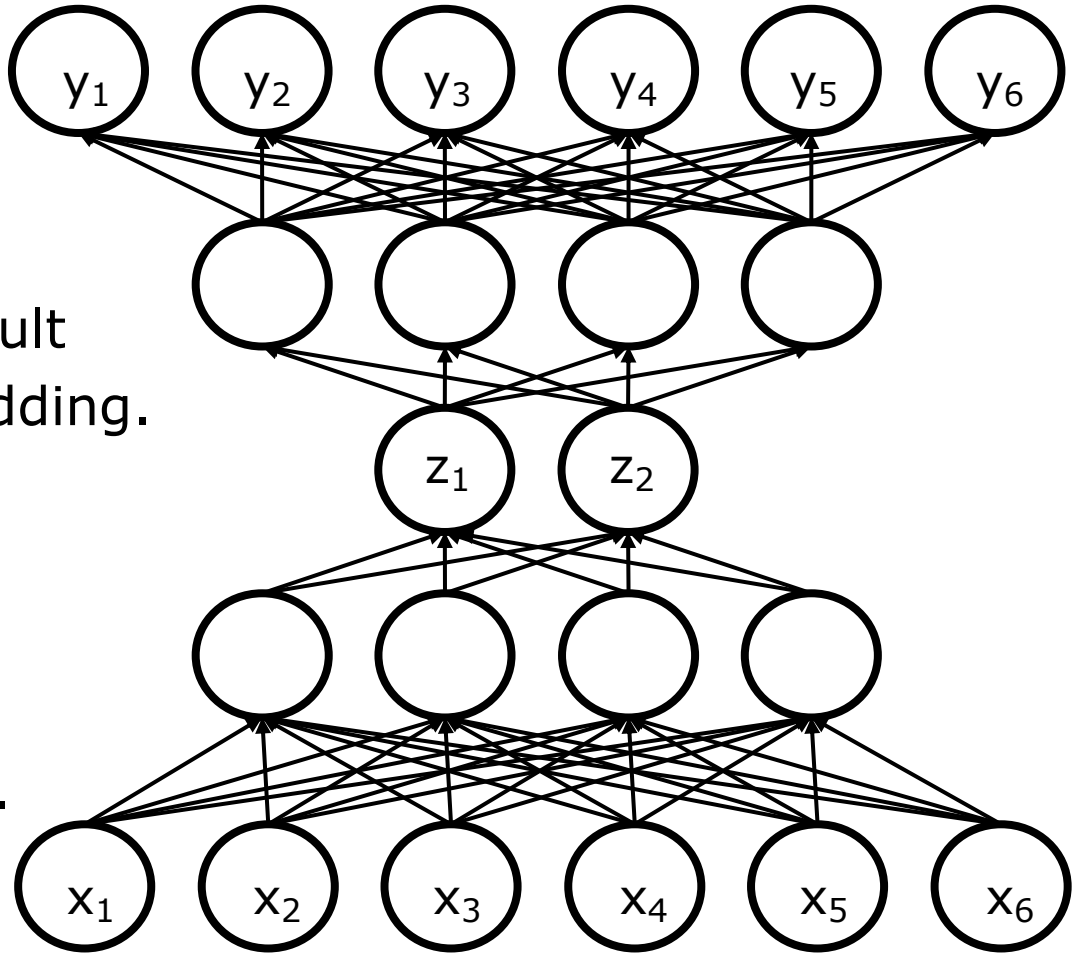


Sensitivity

- Make insensitive:

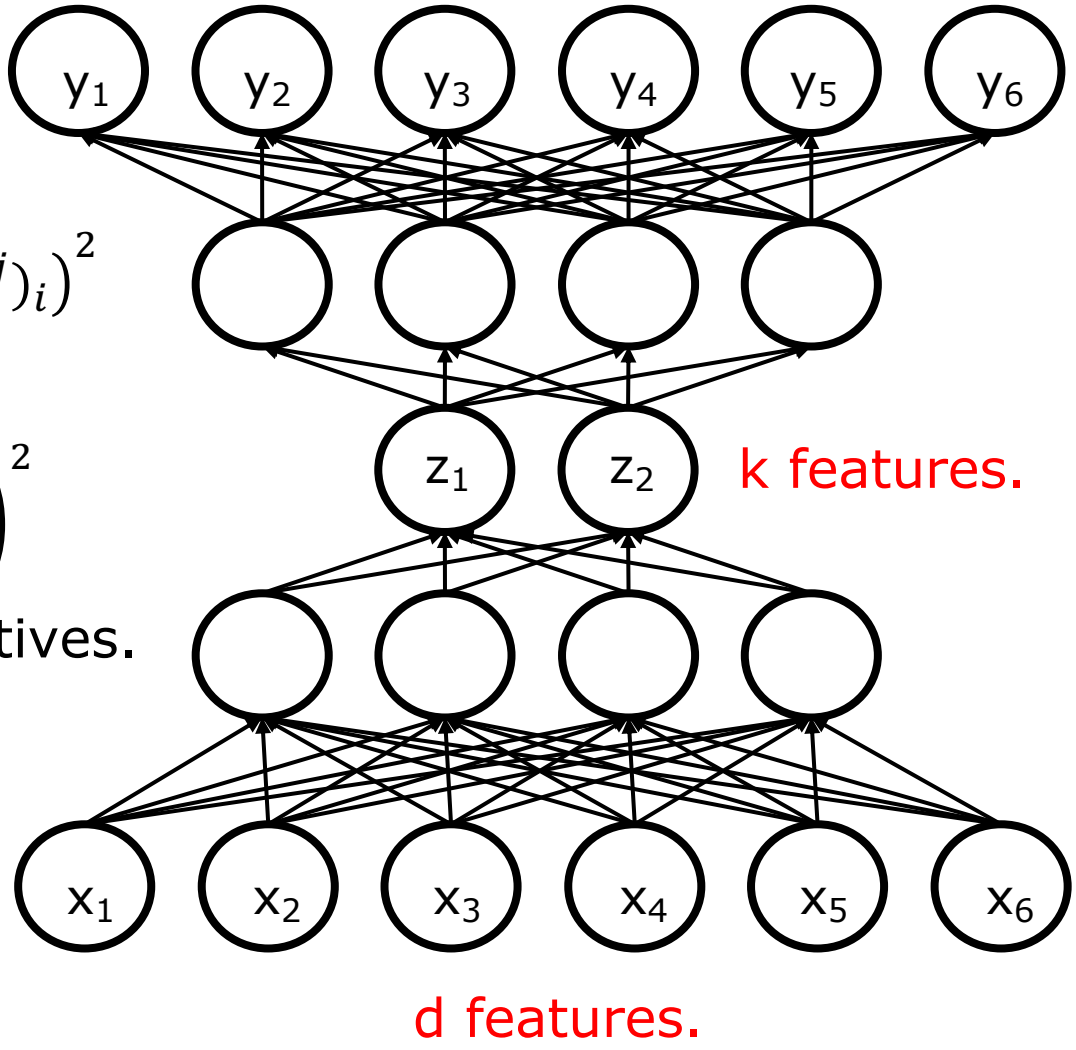
- Sensitive means what?
- Small changes in input result in large changes in embedding.

- Mathematically:
  - Sensitive when partial derivatives of z's wrt. x's are large at inputs.



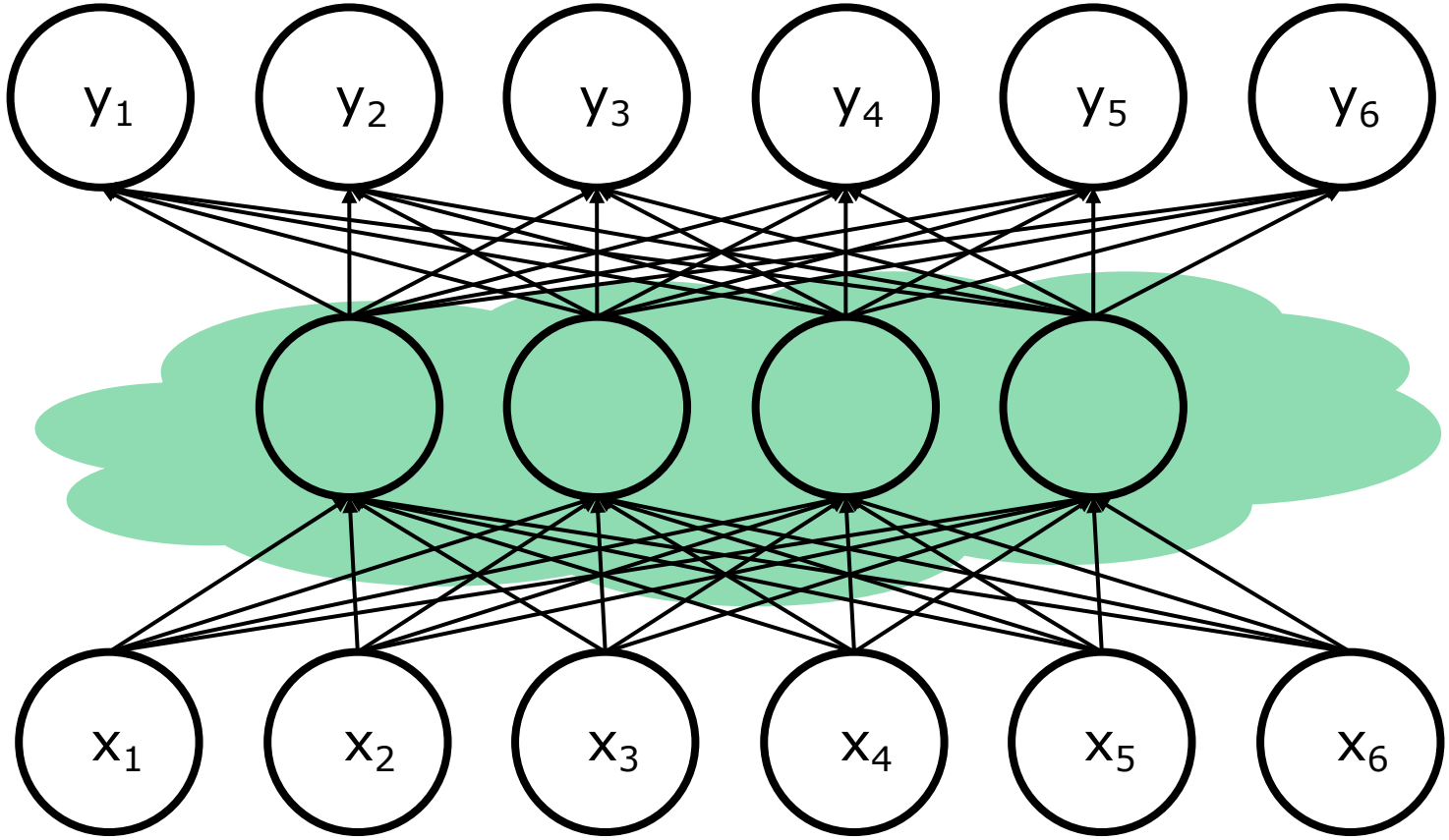
Regularization

- When training:
- Old loss:
  - $L = \frac{1}{n}\sum_{j=1}^{n}\sum_{i=1}^{d}\left(x_i^j - ae(x^j)_i\right)^2$
- New loss:
  - $L + \lambda\sum_{j=1}^{n}\sum_{i=1}^{k}\sum_{h=1}^{d}\left(\frac{\partial z_i}{\partial x_h^j}\right)^2$
- Forces small partial derivatives.
- Use mini-batch SGD.
- λ is hyperparameter.
  - Use validation to try different settings.



Autoencoders

- Neural Nets for Embeddings.



Random Projections

Johnson-Lindenstrauss Note

AARHUS
UNIVERSITY

Dimensionality Reduction

- Redundant Measurements:
  - Data may be redundant:
    - Car: Speed Mph, Fuel, Color, Seats, Speed km/h
  - Or highly correlated:
    - Student: Work Ethics, Grade



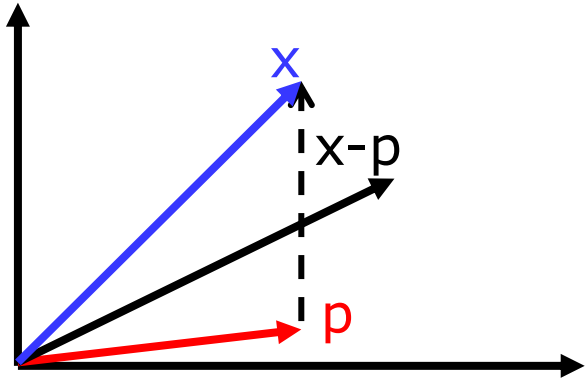
- Goal:
  - Remove redundancies.
  - Reduces number of features.
    - Speeds up/helps algorithms.
    - Visualization if very few features (e.g. 2 or 3).
    - Remove noise.

PCA

- Principal Component Analysis
  - Let $z_1,\ldots,z_k$ be k eigenvectors corresponding to largest eigenvalues of X^TX and project feature vector x to $p_x = \sum_{i=1}^{k}(x^T z_i)z_i$.
- Observation:
  - This choice of $z_1,\ldots,z_k$ minimizes "squared loss" of projections:
    - $\sum_{j=1}^{n} \left\| x_j - p_{x_j} \right\|^2$
- Write x in the new basis:
  - $\hat{x} = \begin{pmatrix} x^T z_1 \\ \vdots \\ x^T z_k \end{pmatrix}$

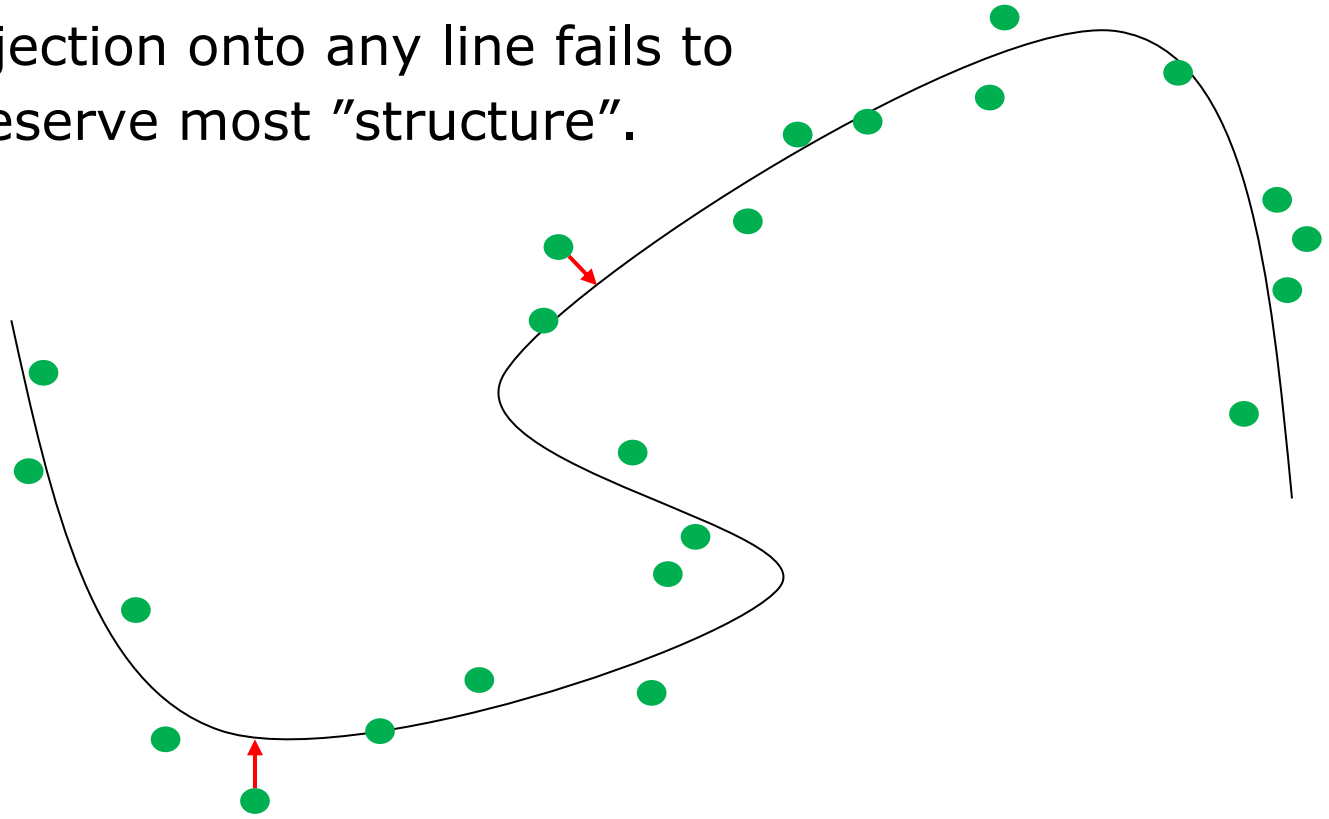


MNIST



PCA Limitations

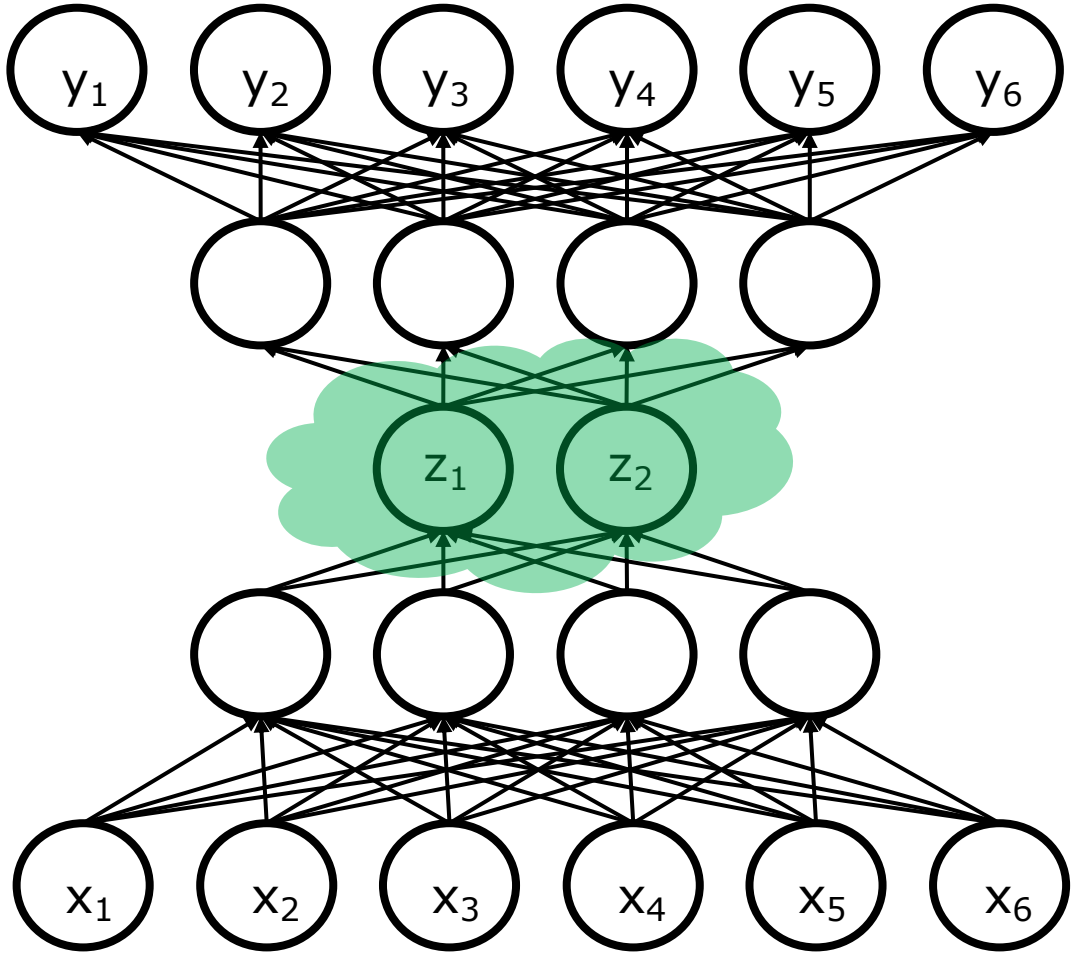
- Embedding is linear
  - Projection onto any line fails to preserve most ”structure”.



  - Non-linear embedding could be useful. Map to position along curve!

Autoencoders

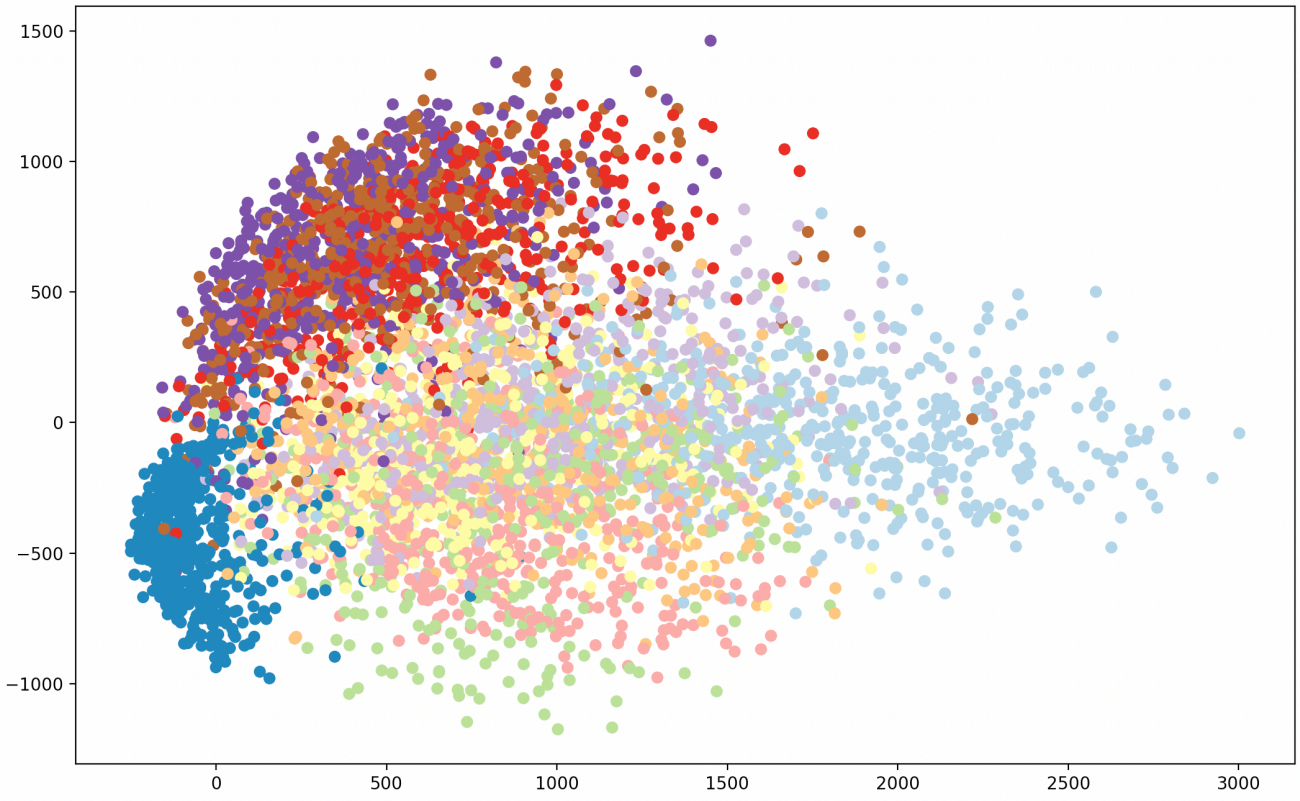
- Neural Nets for embeddings



Limitations

- MNIST
- Many different characters map near each other.
- ”Distance” between encoded vectors does not resemble original ”distance”.
- Only optimizes for decoding.

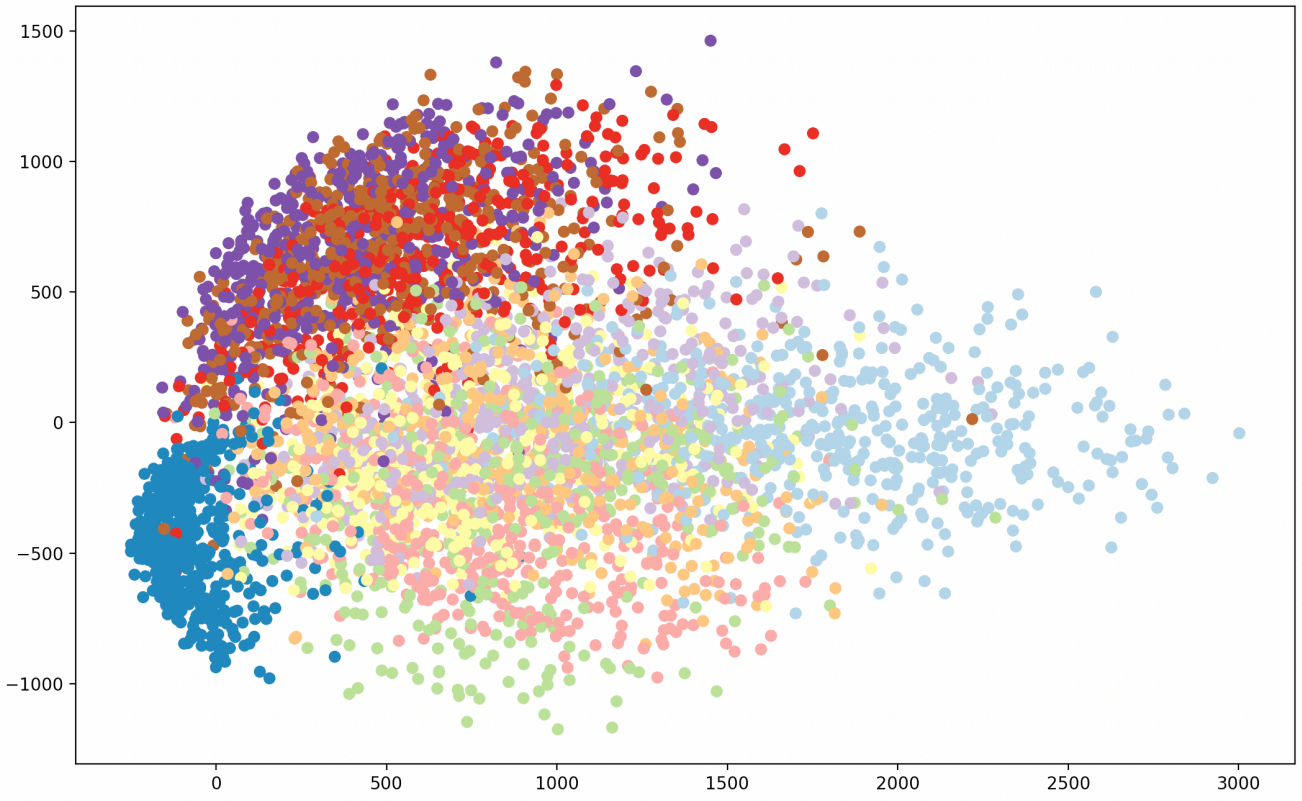
Colors = MNIST classes (digits 0 to 9)



Limitations

- PCA also aims for small loss in reconstruction.
- Does not guarantee preservation of "distance" in embedding.

Colors = MNIST classes (digits 0 to 9)

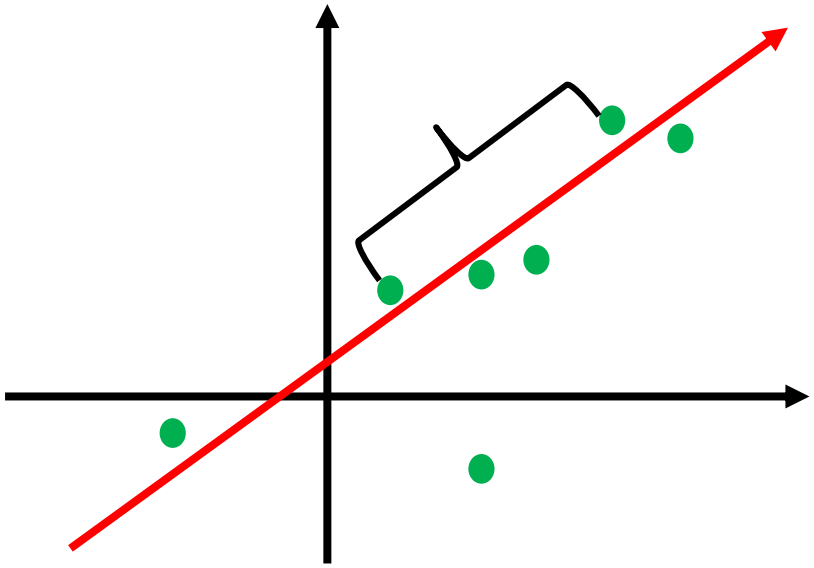


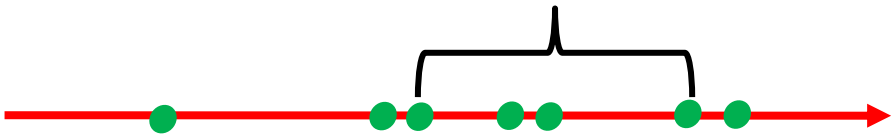
Random Projections

- Johnson-Lindenstrauss Transform
  - Goal: Preserve distances between feature vectors after embedding.



Distance almost the same

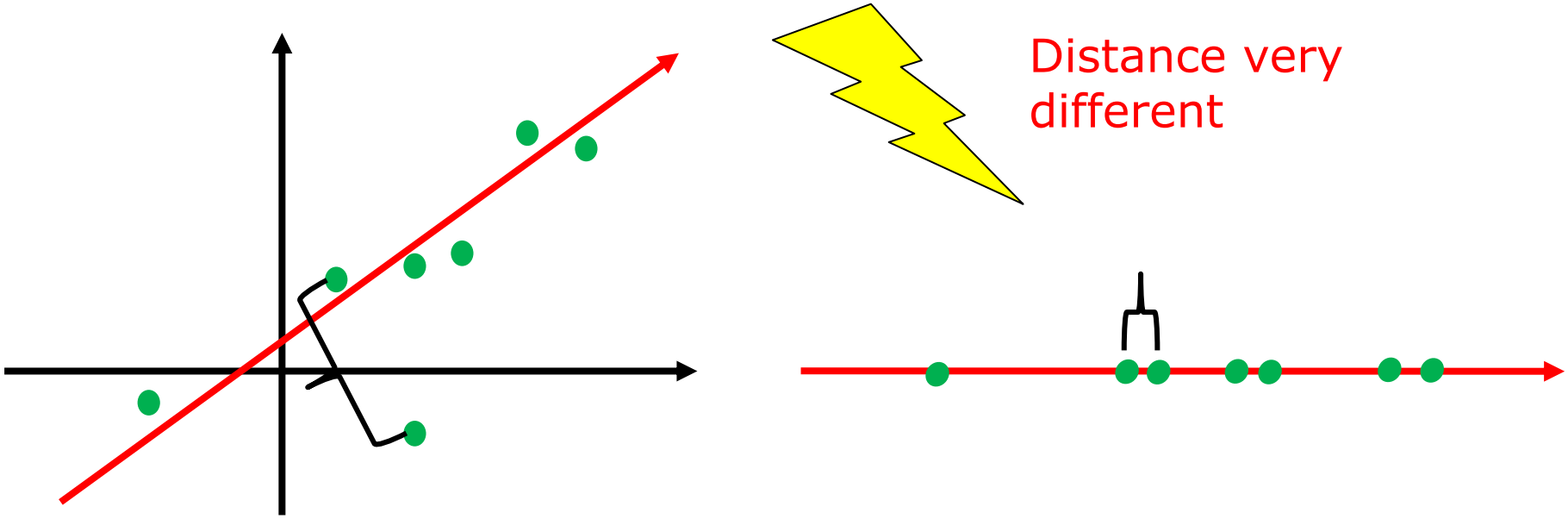




  - Formally: For x_i and x_j, want $\|x_i - x_j\|^2 \approx \|f(x_i) - f(x_j)\|^2$

Random Projections

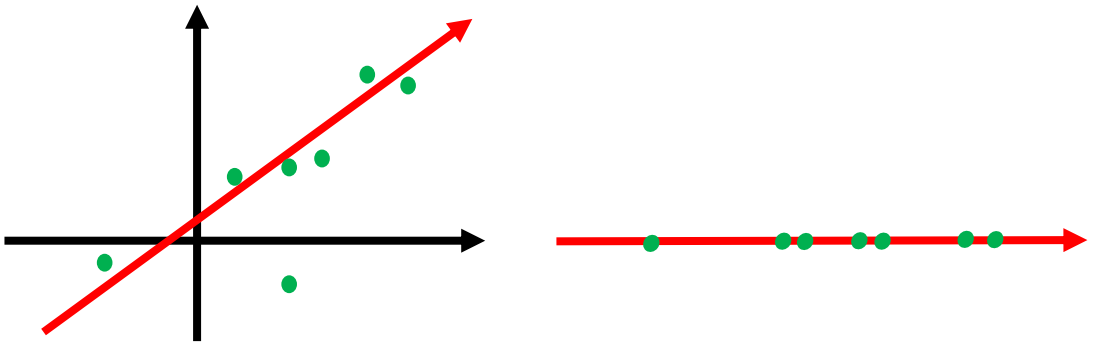
- Johnson-Lindenstrauss Transform
  - Goal: Preserve distances between feature vectors after embedding.



  - Formally: For x_i and x_j, want $\|x_i - x_j\|^2 \approx \|f(x_i) - f(x_j)\|^2$

Random Projections

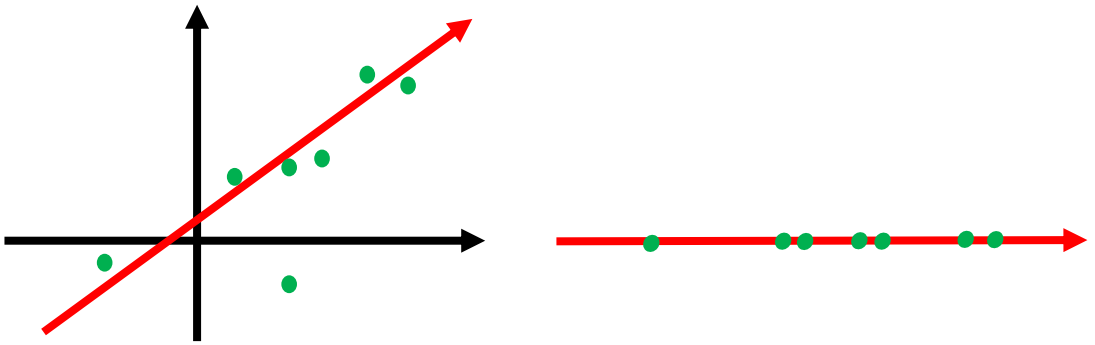
- Johnson-Lindenstrauss Transform
  - Given n feature vectors $x_1,\ldots,x_n$ in R^d.
  - For any precision $0 < \varepsilon < 1/2$ there exists a mapping f : $R^d \to R^k$ with $k = \theta(\varepsilon^{-2} \ln n)$ such that for all x_i, x_j:
    - $(1-\varepsilon)\|x_i - x_j\|^2 \leq \|f(x_i) - f(x_j)\|^2 \leq (1+\varepsilon)\|x_i - x_j\|^2$



- Said in words: Any n feature vectors can be embedded into $\theta(\varepsilon^{-2} \ln n)$ dimensions while preserving all pairwise distances to within a factor $(1 \pm \varepsilon)$.

Random Projections

- Johnson-Lindenstrauss Transform
  - Given n feature vectors $x_1,\ldots,x_n$ in R^d.
  - For any precision $0 < \varepsilon < 1/2$ there exists a mapping f : $R^d \to R^k$ with $k = \theta(\varepsilon^{-2} \ln n)$ such that for all x_i, x_j:
    - $(1-\varepsilon)\|x_i - x_j\|^2 \leq \|f(x_i) - f(x_j)\|^2 \leq (1+\varepsilon)\|x_i - x_j\|^2$



HOW?

- Said in words: Any n feature vectors can be embedded into $\theta(\varepsilon^{-2} \ln n)$ dimensions while preserving all pairwise distances to within a factor $(1 \pm \varepsilon)$.

Random Projections

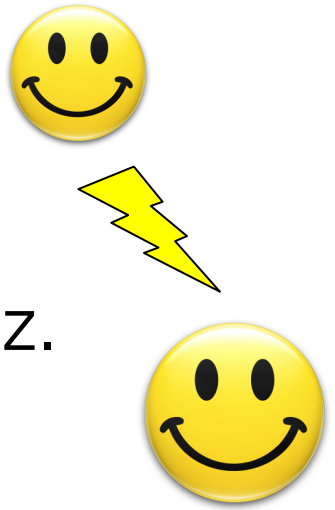
- Johnson-Lindenstrauss Transform
  - Given n feature vectors $x_1,\ldots,x_n$ in R^d.
  - For any precision $0 < \varepsilon < 1/2$ there exists a mapping f : $R^d \to R^k$ with $k = \theta(\varepsilon^{-2} \ln n)$ such that for all x_i, x_j:
    - $(1-\varepsilon)\|x_i - x_j\|^2 \leq \|f(x_i) - f(x_j)\|^2 \leq (1+\varepsilon)\|x_i - x_j\|^2$
    - $X = \begin{pmatrix} - & x_1 & - \\ \vdots & \vdots & \vdots \\ - & x_n & - \end{pmatrix}, Z = \begin{pmatrix} | & \ldots & | \\ z_1 & \ldots & z_k \\ | & \ldots & | \end{pmatrix}$
- Original Construction: Project onto random subspace of dimension $k = \theta(\varepsilon^{-2} \ln n)$ and scale vectors up by $\sqrt{d/k}$.
  - Pick $z_1,\ldots,z_k$ as random unit length and orthogonal vectors. Project data X to $\sqrt{d/k}\,XZ$.



Random Projections

- Johnson-Lindenstrauss: Pick $z_1,\ldots,z_k$ as random unit length and orthogonal vectors. Project data X to $\sqrt{d/k}\,XZ$.
  - $X = \begin{pmatrix} - & x_1 & - \\ \vdots & \vdots & \vdots \\ - & x_n & - \end{pmatrix}, Z = \begin{pmatrix} | & \ldots & | \\ z_1 & \ldots & z_k \\ | & \ldots & | \end{pmatrix}$
- PCA: Pick $z_1,\ldots,z_k$ as the k eigenvectors corresponding to largest eigenvalues of X^TX and project data X to XZ.

- Johnson-Lindenstrauss: Preserves distances.
- PCA: Guarantees best reconstruction if do XZZ^T.
  - Johnson-Lindenstrauss does not.
- Johnson-Lindenstrauss: Much easier to compute Z.
  - Z does not depend on X.
  - Can be chosen before seeing data.



Random Projections

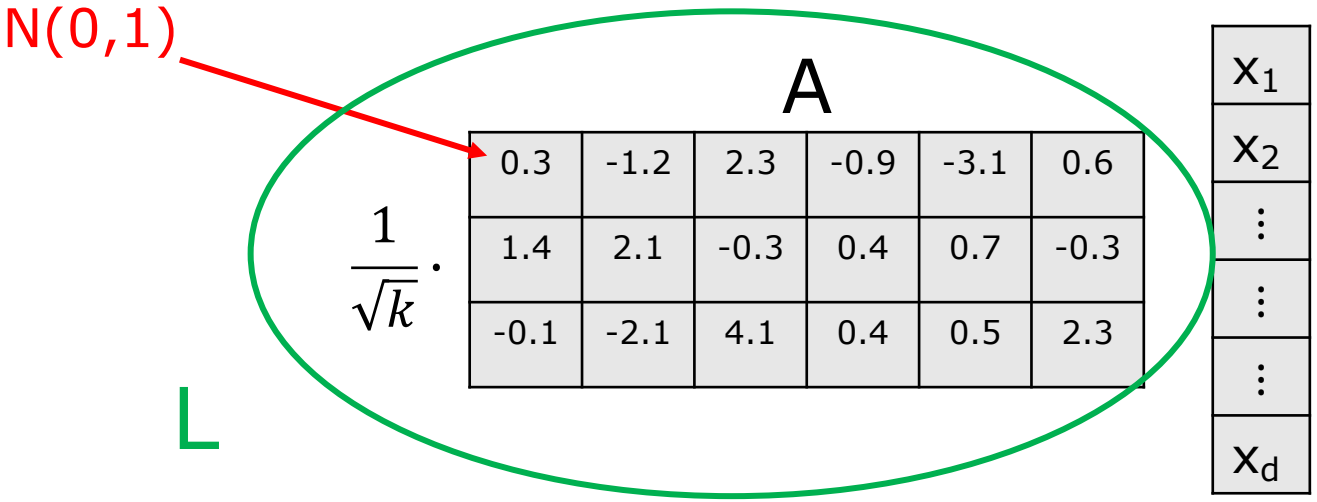
- Johnson-Lindenstrauss: Pick $z_1,\dots,z_k$ as random unit length and orthogonal vectors. Project data X to $\sqrt{d/k}\,XZ$.
  - $X = \begin{pmatrix} - & x_1 & - \\ \vdots & \vdots & \vdots \\ - & x_n & - \end{pmatrix}, Z = \begin{pmatrix} | & \dots & | \\ z_1 & \dots & z_k \\ | & \dots & | \end{pmatrix}$
- Analysis: A little tricky to analyse in this formulation.
- Instead: We analyse a very similar construction. Easier proof.

- Also: Let us go back to data $x_1,\dots,x_n$ as column vectors. Find matrix A and embed as Ax_i. Simply have $Z=A^T$ for above formulation with row vectors in X.



Random Projections

- Johnson-Lindenstrauss Transform
  - Given n feature vectors $x_1,\ldots,x_n$ in R^d.
  - For any precision $0 < \varepsilon < 1/2$ let $L = \frac{1}{\sqrt{k}}A$ where A is k x d with each entry independently N(0,1) distributed and $k = \theta(\varepsilon^{-2} \ln n)$.

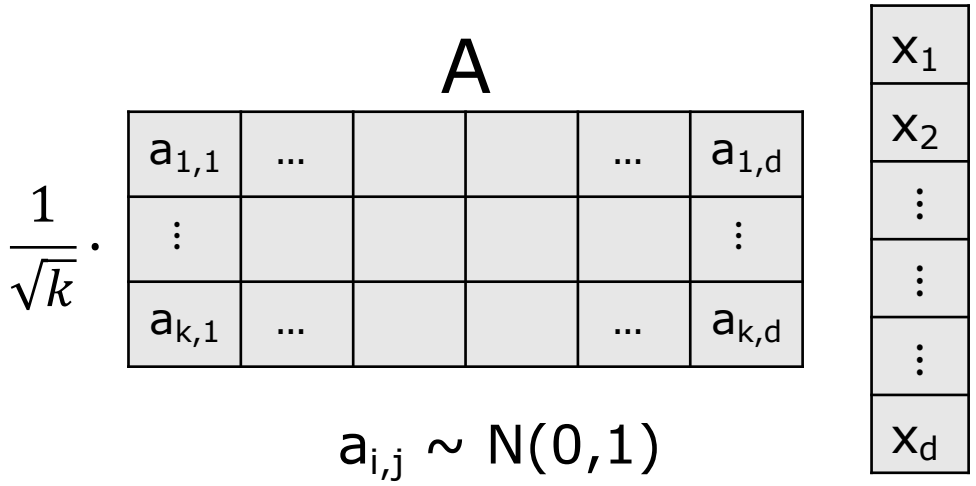


  - With probability at least 1-1/n, it holds for all x_i, x_j:
    - $(1-\varepsilon)\|x_i - x_j\|^2 \leq \|Lx_i - Lx_j\|^2 \leq (1+\varepsilon)\|x_i - x_j\|^2$

Random Projections

$$L = \frac{1}{\sqrt{k}} A$$

- Johnson-Lindenstrauss Transform



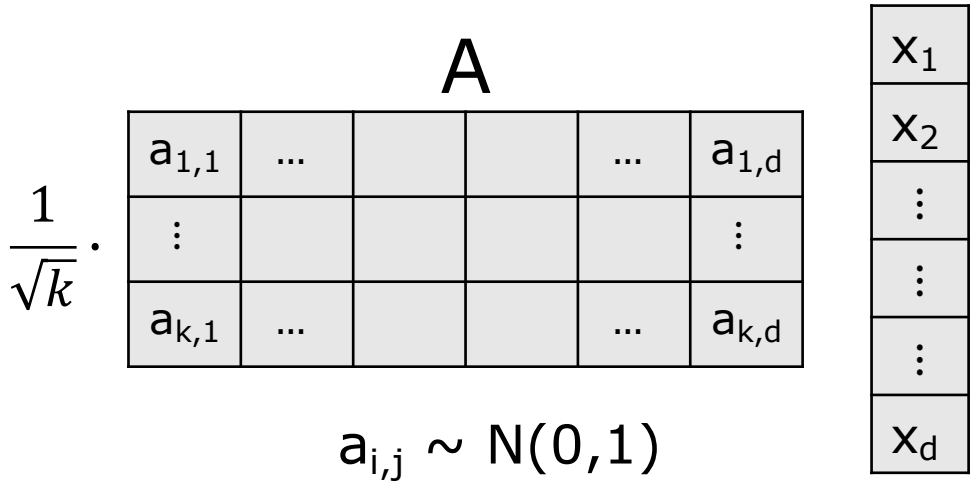
- Want to prove, when $k = \theta(\varepsilon^{-2} \ln n)$:
  - With probability at least 1-1/n, it holds for all x_i, x_j:
    - $(1-\varepsilon)\|x_i - x_j\|^2 \leq \|Lx_i - Lx_j\|^2 \leq (1+\varepsilon)\|x_i - x_j\|^2$



Random Projections

$$L = \frac{1}{\sqrt{k}} A$$

- Johnson-Lindenstrauss Transform



The norm of x is preserved.

- Proof Strategy:
  - Step 1, show when $k = \theta(\varepsilon^{-2} \ln \frac{1}{\delta})$, it holds for any unit vector x
    - $\Pr\left[\left|\|Lx\|^2 - 1\right| \geq \varepsilon\right] \leq \delta$.
  - Step 2, set $\delta = 1/n^3$ and conclude all pairwise distances between vectors $x_1,\ldots,x_n$ are preserved with prob. $\geq 1 - 1/n$.



Random Projections

- Proof Strategy:
  - Step 1, show when $k = \theta(\varepsilon^{-2} \ln \frac{1}{\delta})$, it holds for any unit vector x
    - $\Pr\left[\left|\|Lx\|^2 - 1\right| \geq \varepsilon\right] \leq \delta$.
  - Step 2, set $\delta = 1/n^3$ and conclude all pairwise distances between vectors $x_1,\ldots,x_n$ are preserved with prob. $\geq 1 - 1/n$.

- Assume Step 1 has been done. Step 2:
  - Define event

$$E_{i,j} = \{\|Lx_i - Lx_j\|^2 \notin (1 \pm \varepsilon)\|x_i - x_j\|^2\}$$

  - Want to show:

$$\Pr[E_{1,2} \textbf{ or } E_{1,3} \textbf{ or } \ldots \textbf{ or } E_{n-1,n}] \leq 1/n.$$

$$\Pr[\overline{E_{1,2} \textbf{ or } E_{1,3} \textbf{ or } \ldots \textbf{ or } E_{n-1,n}}] \geq 1\text{-}1/n.$$

The distance is NOT preserved.

There is at least 1 distance that is NOT preserved.

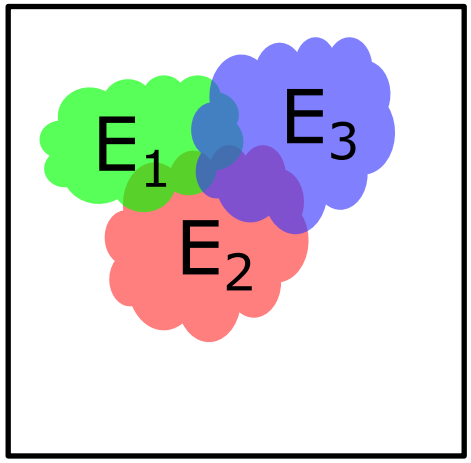
All distances preserved.



Random Projections

- How do we handle all the or's?
  - Union bound!
  - $\Pr[E_1 \text{ or } E_2 \text{ or } \dots \text{ or } E_M] \leq \Pr[E_1]+\Pr[E_2]+\dots+\Pr[E_M]$.

Proof by picture



Random Projections

- Define event

$$E_{i,j} = \{\|Lx_i - Lx_j\|^2 \notin (1 \pm \varepsilon)\|x_i - x_j\|^2\}$$

$$\Pr[E_{1,2} \textbf{ or } E_{1,3} \textbf{ or } \dots \textbf{ or } E_{n-1,n}] \leq \sum_{i<j} \Pr[E_{i,j}].$$

Define $v = \frac{x_i - x_j}{\|x_i - x_j\|}$. Then v unit length and:

$$\{\|Lx_i - Lx_j\|^2 \notin (1 \pm \varepsilon)\|x_i - x_j\|^2\} =$$
$$\{\|L(x_i - x_j)\|^2 \notin (1 \pm \varepsilon)\|x_i - x_j\|^2\} =$$
$$\{\|Lv\|x_i - x_j\|\|^2 \notin (1 \pm \varepsilon)\|v\|x_i - x_j\|\|^2\} =$$
$$\{\|x_i - x_j\|^2\|Lv\|^2 \notin (1 \pm \varepsilon)\|x_i - x_j\|^2\} =$$
$$\{\|Lv\|^2 \notin (1 \pm \varepsilon)\}$$

L fails to preserve the distance between x_i and x_j if and only if it fails to preserve the norm of v.



Random Projections

- Proof Strategy:
  - Step 1, show when $k = \theta(\varepsilon^{-2} \ln \frac{1}{\delta})$, it holds for any unit vector x
    - $\Pr\left[\left|\|Lx\|^2 - 1\right| \geq \varepsilon\right] \leq \delta$.
  - Step 2, set $\delta = 1/n^3$ and conclude all pairwise distances between vectors $x_1,\ldots,x_n$ are preserved with prob. $\geq 1 - 1/n$.

- Assume Step 1 has been done. Step 2:
  - Define event

$$E_{i,j} = \{\|Lx_i - Lx_j\|^2 \notin (1 \pm \varepsilon)\|x_i - x_j\|^2\}$$

$$\Pr[E_{1,2} \textbf{ or } E_{1,3} \textbf{ or } \ldots \textbf{ or } E_{n-1,n}] \leq \sum_{i<j} \Pr[E_{i,j}].$$

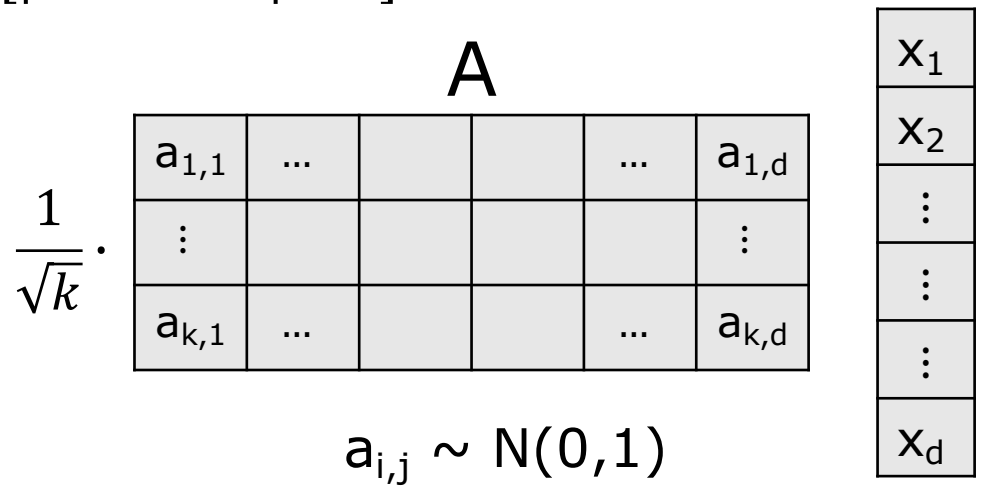
$$\Pr[E_{i,j}] = \Pr[\|Lv\|^2 \notin (1 \pm \varepsilon)] \leq \delta = 1/n^3.$$

$$\Pr[E_{1,2} \textbf{ or } E_{1,3} \textbf{ or } \ldots \textbf{ or } E_{n-1,n}] \leq \sum_{i<j} \Pr[E_{i,j}] < 1/n.$$



Random Projections

- What Remains:
  - Step 1, show when $k = \theta(\varepsilon^{-2} \ln \frac{1}{\delta})$, it holds for any unit vector x
    - $\Pr\left[\left|\|Lx\|^2 - 1\right| \geq \varepsilon\right] \leq \delta$.



$$L = \frac{1}{\sqrt{k}} A$$

  - Coordinate i of Ax:
    - $\sum_{j=1}^{d} a_{i,j} x_j$

Constant x_j times N(0,1) distributed.

Normal Distribution

- Two Useful Properties:
  - Let a be N(0,1) distributed and let c be a constant.
    - $c \cdot a \sim N(0, c^2)$
  - Let $x \sim N(0, \sigma_x^2)$ and $y \sim N(0, \sigma_y^2)$ be independent.
    - $x+y \sim N(0, \sigma_x^2 + \sigma_y^2)$.



Random Projections

- What Remains:
  - Step 1, show when $k = \theta(\varepsilon^{-2} \ln \frac{1}{\delta})$, it holds for any unit vector x
    - $\Pr\left[\left|\|Lx\|^2 - 1\right| \geq \varepsilon\right] \leq \delta$.

$$\frac{1}{\sqrt{k}} \cdot \begin{array}{|c|c|c|c|c|c|} \hline a_{1,1} & \dots & & & \dots & a_{1,d} \\ \hline \vdots & & & & & \vdots \\ \hline a_{k,1} & \dots & & & \dots & a_{k,d} \\ \hline \end{array} \quad \begin{array}{|c|} \hline x_1 \\ \hline x_2 \\ \hline \vdots \\ \hline \vdots \\ \hline \vdots \\ \hline x_d \\ \hline \end{array}$$

A

$a_{i,j} \sim N(0,1)$

$$L = \frac{1}{\sqrt{k}} A$$

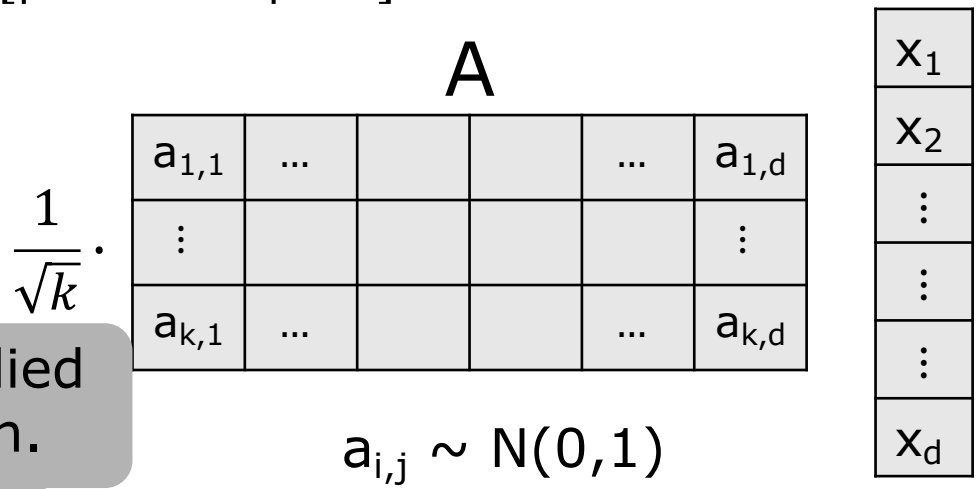
$a_{i,j} x_j \sim N(0, x_j^2)$

  - Coordinate i of Ax:
    - $\sum_{j=1}^{d} a_{i,j} x_j \sim N(0, \sum_{j=1}^{d} x_j^2) = N(0, \|x\|^2) = N(0,1)$



Random Projections

- What Remains:
  - Step 1, show when $k = \theta(\varepsilon^{-2} \ln \frac{1}{\delta})$, it holds for any unit vector x
    - $\Pr\left[\left|\|Lx\|^2 - 1\right| \geq \varepsilon\right] \leq \delta$.



$$L = \frac{1}{\sqrt{k}} A$$

The well-studied χ^2_k distribution.

- Each coordinate of Ax is independently N(0,1) distributed.
- $\|Ax\|^2$ distributed as the sum of k squared indep. N(0,1) vars.

Pronounced chi-squared distribution with k degrees of freedom.

Random Projections

- What Remains:
  - Step 1, show when $k = \theta(\varepsilon^{-2} \ln \frac{1}{\delta})$, it holds for any unit vector x
    - $\Pr\left[\left|\|Lx\|^2 - 1\right| \geq \varepsilon\right] \leq \delta$.

$$\frac{1}{\sqrt{k}} \cdot \mathrm{A} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ \vdots \\ \vdots \\ x_d \end{pmatrix}$$

$a_{1,1}$	...			...	$a_{1,d}$
⋮					⋮
$a_{k,1}$	...			...	$a_{k,d}$

$a_{i,j} \sim N(0,1)$

$$L = \frac{1}{\sqrt{k}} \mathrm{A}$$

  - $\|Ax\|^2 \sim \chi_k^2$
  - $\|Lx\|^2 = \frac{\|Ax\|^2}{k} \sim \frac{\chi_k^2}{k}$



Random Projections

- What Remains:
  - Step 1, show when $k = \theta(\varepsilon^{-2} \ln \frac{1}{\delta})$, it holds for any unit vector x
    - $\Pr\left[\left|\|Lx\|^2 - 1\right| \geq \varepsilon\right] \leq \delta$.
    - $\|Lx\|^2 = \frac{\|Ax\|^2}{k} \sim \frac{\chi_k^2}{k}$

- Tail-bound (like Hoeffding's inequality from earlier):
  - Let $Y \sim \chi_k^2$ then
    - $\Pr\left[\left|\frac{Y}{k} - 1\right| \geq \varepsilon\right] \leq e^{-\frac{3}{16}k\varepsilon^2}$ for any $0 < \varepsilon < \frac{1}{2}$
  - $\Pr\left[\left|\|Lx\|^2 - 1\right| \geq \varepsilon\right] \leq e^{-\frac{3}{16}C\varepsilon^{-2}\ln\frac{1}{\delta}\varepsilon^2} = e^{-\frac{3}{16}C\ln\frac{1}{\delta}} \leq e^{-\ln\frac{1}{\delta}} = \delta$

    (assume $C \geq \frac{16}{3}$ big enough constant in $k = C\varepsilon^{-2}\ln\frac{1}{\delta}$).



Random Projections

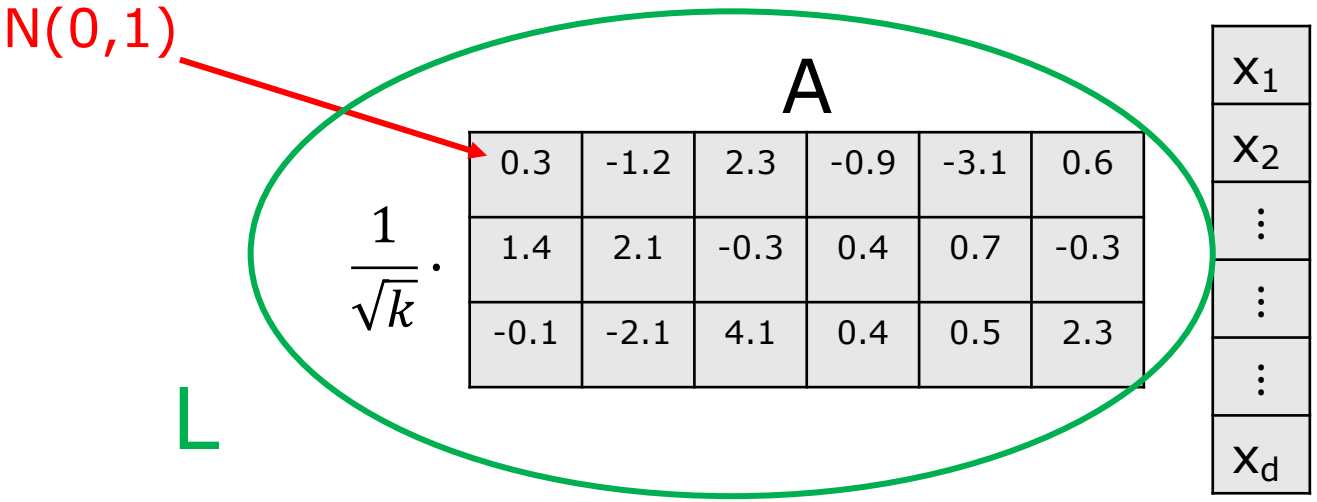
- Proof Strategy:
  - Step 1, show when $k = \theta(\varepsilon^{-2} \ln \frac{1}{\delta})$, it holds for any unit vector x
    - $\Pr\left[\left|\|Lx\|^2 - 1\right| \geq \varepsilon\right] \leq \delta$.
  - Step 2, set $\delta = 1/n^3$ and conclude all pairwise distances between vectors $x_1,\ldots,x_n$ are preserved with prob. $\geq 1 - 1/n$.

- Conclusion:
  - Works when $k = \theta\left(\varepsilon^{-2} \ln \frac{1}{\delta}\right) = \theta(\varepsilon^{-2} \ln n^3) = \theta(\varepsilon^{-2} \ln n)$



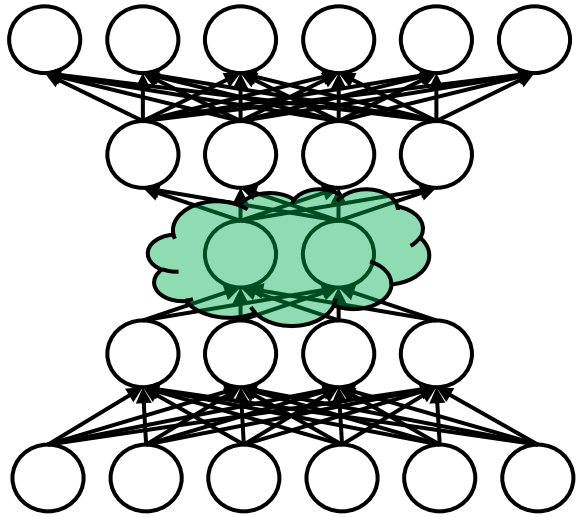
Random Projections

- Johnson-Lindenstrauss Transform
  - Given n feature vectors $x_1,\ldots,x_n$ in R^d.
  - For any precision $0 < \varepsilon < 1/2$ let $L = \frac{1}{\sqrt{k}}A$ where A is k x d with each entry independently N(0,1) distributed and $k = \theta(\varepsilon^{-2} \ln n)$.



  - With probability at least 1-1/n, it holds for all x_i, x_j:
    - $(1-\varepsilon)\|x_i - x_j\|^2 \leq \|Lx_i - Lx_j\|^2 \leq (1+\varepsilon)\|x_i - x_j\|^2$

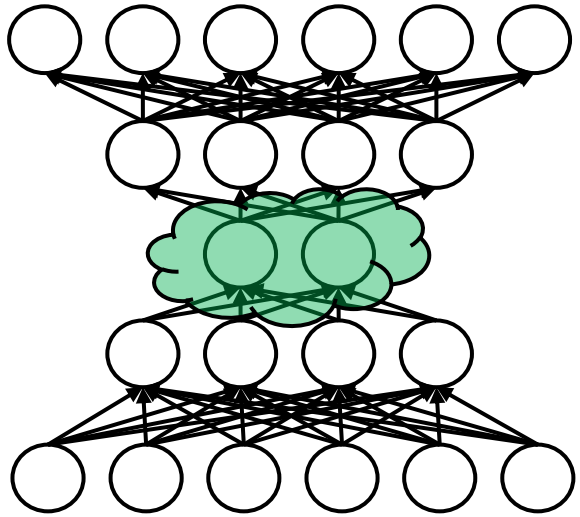
Last Remarks



- Johnson-Lindenstrauss Transform
  - Embed into $k = \theta(\varepsilon^{-2} \ln n)$ dimensions.
- Natural Question:
  - Might we embed into fewer dimensions?
  - Johnson-Lindenstrauss is a linear embedding, might non-linear embeddings help (like neural nets)?
- Answer:
  - No, not in general.
  - Larsen, Nelson'17: There exists vectors $x_1,\ldots,x_n$ in R^d such that *any* embedding preserving distances to $(1 \pm \varepsilon)$ must embed into $k = \Omega(\varepsilon^{-2} \ln n)$.



Last Remarks



- Johnson-Lindenstrauss Transform
  - Embed into $k = \theta(\varepsilon^{-2} \ln n)$ dimensions.

- Extensions:
  - A has entries N(0,1) distributed.
  - May be tedious to implement.
  - It also works if each entry is chosen uniformly among {-1,1}.

Projections

- Johnson-Lindenstrauss: Pick $z_1,\ldots,z_k$ as random unit length and orthogonal vectors. Project data X to $\sqrt{d/k}\,XZ$.
  - $X = \begin{pmatrix} - & x_1 & - \\ \vdots & \vdots & \vdots \\ - & x_n & - \end{pmatrix}, Z = \begin{pmatrix} | & \ldots & | \\ z_1 & \ldots & z_k \\ | & \ldots & | \end{pmatrix}$
- PCA: Pick $z_1,\ldots,z_k$ as the k eigenvectors corresponding to largest eigenvalues of X^TX and project data X to XZ.

- Johnson-Lindenstrauss: Preserves distances.
- PCA: Guarantees best reconstruction if do XZZ^T.
  - Johnson-Lindenstrauss does not.
- Johnson-Lindenstrauss: Much easier to compute Z.
  - Z does not depend on X.
  - Can be chosen before seeing data.

